LLM Fundamentals

A field guide I put together for engineers new to AI — I've answered, one by one, the questions you asked during our sessions. It assumes you write software but have never trained a neural network.

21 SECTIONS · A–U · PLAIN-LANGUAGE ANSWERS WITH DIAGRAMS, WORKED EXAMPLES & SAMPLE MATRICES

Compiled by **Amit Pandey** · amit.s.pandey@gmail.com · linkedin.com/in/amitpandeyprofile

HOW TO READ THIS GUIDE

I've tried to leave you with a *mental model you can reason with* in every answer, not just a definition. So most answers carry one or more of these labelled boxes. When you see a box, here's what I'm doing with it:

> **ANALOGY**
> A familiar, non-AI comparison to anchor the idea before any jargon. Read this first if a concept feels abstract.

> **WORKED EXAMPLE**
> A concrete, end-to-end walk-through with real-ish numbers or text, so you can see the mechanism actually run rather than just hear it described.

> **COMMON MISCONCEPTION**
> A belief many people hold that is subtly or completely wrong. Several of your questions were built on one of these — we call it out and correct it.

> **IN ONE SENTENCE**
> *A single line you could say out loud to explain the idea to a teammate — the compressed takeaway.*

> **KEY TAKEAWAY**
> The practical consequence for you as a builder — what to actually do, choose, or avoid.

Many answers also include a **sample matrix** (a small table you can scan) and a **diagram** (boxes and arrows). I've kept each question in its original wording with the name of whoever asked it; where a few of you asked variants of the same thing, I merged them and credited everyone.

CONTENTS

A Training & Learning

Let me start at the beginning: how a model goes from random numbers to something useful, and what I actually mean by "learning" and "size."

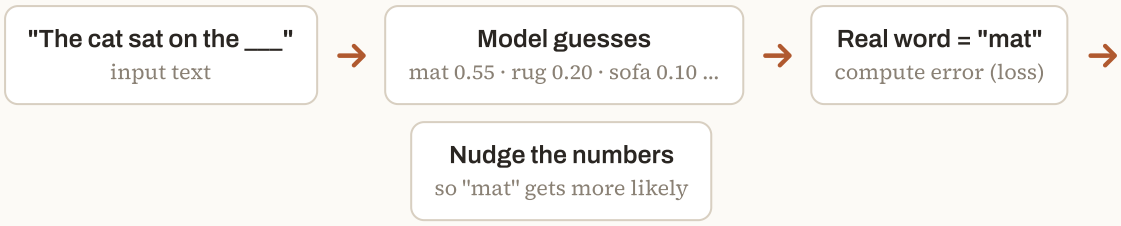
A1 How do LLMs learn, and is there a single fixed way to train them?

Ajay Sarswat

There is **no single fixed recipe**, but almost every modern LLM goes through the same three-stage pattern. The engine underneath is dead simple and never changes: *the model repeatedly tries to predict the next word in a piece of text, checks whether it was right, and nudges its internal numbers to be slightly less wrong next time.* Do that a few trillion times and language patterns get baked into those numbers.

> **ANALOGY**
> Imagine someone who has never spoken English is handed millions of books with the last word of every sentence hidden. They guess each hidden word, peek at the real answer, and adjust their instincts. After enough books they don't "know facts" the way a database does — they've developed a feel for what word tends to come next. That feel *is* the model.

Amit Pandey · amit.s.pandey@gmail.com



The training loop. Repeat billions of times over huge amounts of text. This one loop, called **self-supervised next-token prediction**, is the whole of "pretraining."

After pretraining, the raw model can complete text but isn't yet a helpful assistant. Two more stages shape its behaviour:

SAMPLE MATRIX — THE THREE STAGES OF TRAINING

STAGE	WHAT IT TEACHES	DATA IT USES	ANALOGY
1. Pretraining	Grammar, facts, reasoning patterns, world structure	Trillions of tokens of raw internet/book/code text	Reading the entire library
2. Supervised fine-tuning (SFT)	How to follow an instruction and format an answer	Curated `prompt → ideal answer` pairs written by humans	Studying worked examples from a tutor
3. Preference tuning (RLHF / DPO)	Which answer humans prefer — helpful, honest, safe tone	Humans ranking two model answers "A is better than B"	Being coached on judgement and tone

COMMON MISCONCEPTION

"Training is one monolithic process." In practice it's a pipeline, and the later stages are where a model gets its personality and guardrails. Two labs can start from an almost identical pretrained base and end up feeling completely different purely because of stages 2 and 3.

IN ONE SENTENCE

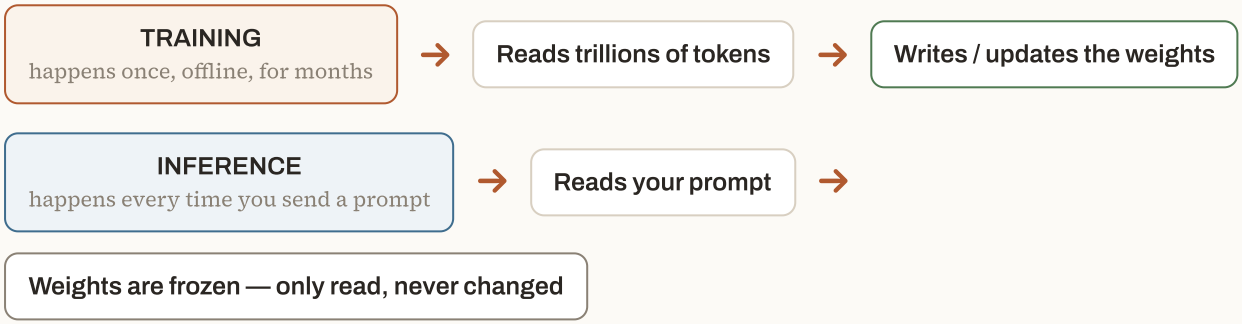
"An LLM learns by guessing the next word in real text billions of times and correcting itself, then gets fine-tuned and coached with human feedback — the guessing engine is fixed, but the coaching recipe varies from lab to lab."

A2 How do LLMs retain / store information from training data, and how is that learning used later at inference?

rohit Chauhan · Amit Manchanda

The knowledge is stored in the **parameters** (also called weights) — the billions of numbers inside the network. Training gradually adjusts these numbers so that useful patterns become encoded in them. Crucially, the model does **not** keep a copy of the training documents it can look through. The text is gone; only its statistical residue, spread across billions of weights, remains.

> **ANALOGY**
>
> Training is like *baking a cake*. Flour, eggs and sugar (the training text) go in, and after baking you get a cake (the weights). You cannot pull the individual eggs back out of the finished cake — and you don't need to. The flavour (the knowledge) is now distributed throughout. Inference is just *slicing and serving* that cake; it doesn't change the recipe.



Two different phases. Learning writes the weights (top). Answering you only reads them (bottom). Nothing you type during a chat is written back into the model.

SAMPLE MATRIX — TRAINING VS INFERENCE

	TRAINING	INFERENCE (USING IT)
WHEN	Once, before release (weeks–months)	Every single prompt, in milliseconds–seconds
DO THE WEIGHTS CHANGE?	Yes — that's the whole point	No — read-only
DOES IT "LEARN" FROM YOUR CHAT?	—	No. It forgets everything once the request ends
COST DRIVER	Enormous one-time compute	Compute per token generated

So "how is the learning used at inference?" — when you send a prompt, the model runs a **forward pass**: your words flow through those frozen weights, and the same patterns learned in training produce a probability for the next word. The knowledge isn't retrieved from a table; it's re-computed on the fly from the weights every time.

Amit Pandey · amit.s.pandey@gmail.com

IN ONE SENTENCE

"Everything the model 'knows' lives baked into its billions of weights, not in a searchable copy of the data — and at inference it just reads those frozen weights to compute the next word, learning nothing new from you."

A3 When a model has 8B / 70B parameters, what do those numbers represent, and how are the parameters chosen out of billions?

rohit Chauhan | Shubh Mandalia

"8B" means the network contains **8 billion individual numbers** (weights and biases). Each is a tiny dial that controls how strongly one internal signal influences another. They are **not selected from a larger pool** — every one of the 8 billion is used, and every one is *learned*. The count is a design decision made *before* training; then training tunes all of them at once.

COMMON MISCONCEPTION

The phrase "chosen out of billions" suggests the model picks a useful few and discards the rest. That's not what happens. All 8 billion are active. Training doesn't *choose* parameters — it *tunes* the value of every parameter, starting from random noise and adjusting each one slightly on every step.

WORKED EXAMPLE

Shrink it to a 3-parameter "model" that predicts a house price:

```
price = w1·(size) + w2·(bedrooms) + b
```

Here `w1`, `w2` and `b` are the parameters. Training starts them at random (say `0.1, 0.1, 0`), predicts a price, sees the real price, and nudges each dial to reduce the error. After many examples they settle at, say, `w1=210, w2=15000, b=30000`. An 8B model is exactly this — just with 8 billion dials arranged in layers, tuning things far more abstract than "bedrooms."

Amit Pandey · amit.s.pandey@gmail.com

SAMPLE MATRIX — WHAT THE PARAMETER COUNT BUYS YOU

PARAMS	ROUGH MEMORY TO RUN (16-BIT)	TYPICAL USE	FEEL
~1–3B	2–6 GB	On-phone, edge devices	Fast, forgetful, basic
8B	~16 GB (one good GPU)	Local dev, cheap tasks	Capable for its size
70B	~140 GB (multi-GPU)	Strong general assistant	Noticeably smarter, slower, costlier
Frontier (est. 100s of B–T)	Data-centre scale	Best reasoning	State of the art

More parameters ≈ more capacity to store patterns (roughly like more neurons), which usually means smarter output — but also more memory, more cost, and more latency. Bigger is not automatically better; a well-trained 8B can beat a poorly-trained 70B.

> **IN ONE SENTENCE**
>
> *"8B means eight billion tunable dials inside the network — none are 'picked out,' they're all used and all learned, and the number is a capacity choice made before training ever starts."*

A4 If an LLM is trained on much more data over ~9 months, how big does it get, and how does it "hold" the context of all that data?

Thirukumaran K

This is the single most useful distinction to get straight, because two very different "sizes" are being blended together here:

- **Model size** — the parameter count (8B, 70B). This is *fixed by the architecture before training* and does **not** grow just because you feed in more data. Train an 8B model on 10× more text for 9 months and it is still an 8B model on disk.
- **Context window** — how much text the model can look at *in a single request* at inference time (e.g. 128K or 1M tokens). This is a runtime limit, unrelated to how much data it was trained on.

Amit Pandey · amit.s.pandey@gmail.com

ANALOGY

Think of a chef. **Training data** is every meal they ever cooked or tasted over their career — it shaped their skill but you can't list those meals. Their **skill level** (model size) is fixed capacity; cooking more meals sharpens it but doesn't give them a bigger brain. The **context window** is the recipe card they can glance at *right now* while cooking one dish. Feeding them more career experience never enlarges the recipe card on the counter.

So what does more data over 9 months buy you? Not size — **quality**. The same-size model gets better predictions, fewer gaps, more up-to-date facts. It "holds" the training data the way the chef holds their career: as improved instinct baked into fixed-size weights, not as a stored, browsable archive.

SAMPLE MATRIX — WHAT CHANGES WHEN YOU TRAIN LONGER / ON MORE DATA

PROPERTY	CHANGES WITH MORE TRAINING DATA?	WHY
Parameter count / file size	**No**	Fixed by architecture chosen up front
Answer quality & coverage	**Yes, improves**	Weights are tuned on more examples
How recent its knowledge is	Yes, up to the new cutoff	Newer text was seen (see Section C)
Context window (tokens per request)	No — separate design choice	It's a runtime limit, not a data effect

IN ONE SENTENCE

"Feeding a model more data over months makes an 8B model a better 8B model, not a bigger one — data improves the skill baked into a fixed number of weights; it doesn't enlarge the model or the amount it can read at once."

B Next-Token Prediction & Context

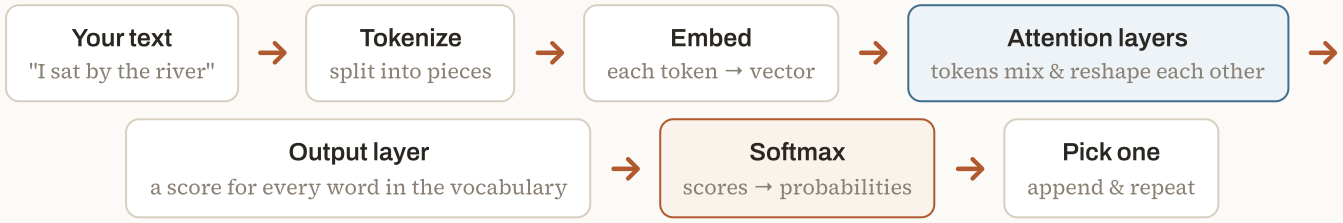
Here I'll show you what physically happens on each step, and what I really mean when I say the model "understands context."

Amit Pandey · amit.s.pandey@gmail.com

B1 What actually happens inside an LLM when it generates the next token? Does the context override the existing probability, and what does "understanding context" really mean?

Jishan Baig · VINAYAK AWASARE

Generating one token is a fixed pipeline called the **forward pass**. Everything you've typed goes in; a probability for every possible next token comes out; one token is picked; then the whole thing repeats with that token appended. Here is the pipeline end-to-end:



One forward pass = one token. A 200-word answer runs this loop ~260 times. The attention layers (blue) are where "context" does its work.

Does context override an existing probability? No — and this is the key correction. There is no pre-existing, context-free probability sitting inside the model waiting to be overridden. The model computes `P(next token | everything in the context)` *from scratch* every time. Context isn't a filter applied afterward; it's the *input to the calculation*. Change the context and you get a different computation, not a modified one.

COMMON MISCONCEPTION

"The word has a base probability, and my prompt tweaks it." Closer to the truth: the word only *has* a probability once you supply context. With no context the model would fall back to raw frequencies of language; your prompt is what turns those into a specific, situation-aware prediction.

WORKED EXAMPLE — SAME BLANK, DIFFERENT CONTEXT

Consider the blank in "...the ___". Watch how the probabilities are recomputed when the earlier words change:

SAMPLE MATRIX — HOW CONTEXT RESHAPES THE NEXT-TOKEN PROBABILITIES

CONTEXT SO FAR	BANK	WATER	MONEY	MAT
"I sat by the river and looked at the ..."	0.31	0.28	0.01	0.00
"I went to the ATM to withdraw some ..."	0.04	0.00	0.71	0.00
"The cat sat on the ..."	0.00	0.00	0.00	0.62

Same target position, wildly different distributions — because each was *computed from* its context, not adjusted from a shared baseline. That is what "understanding context" means mechanically: in the attention layers, the vector for "the" is literally reshaped by "river" or "ATM" before the prediction is made, so the word "bank" carries a different internal meaning in each case.

> **IN ONE SENTENCE**
>
> *"The model doesn't override a stored probability with your context — it computes the next-token probabilities freshly out of your context every step, which is exactly why the same sentence-ending can come out as 'water,' 'money,' or 'mat.'"*

B2 Does the prediction change based on compute resources (CPU)?

Krishna sagar

No. For a *given* model and *given* settings, the prediction is defined by the maths, not by the hardware. Running it on a laptop CPU, a phone, or a rack of GPUs changes **how fast** the answer comes and **how big a model you can afford to run** — but not what the correct next-token probabilities are.

> **ANALOGY**
>
> It's like computing `3572 × 891`. A fast computer and a slow one give the *same* answer; one just returns it sooner. Hardware is the speed of the calculator, not the arithmetic.

Amit Pandey · amit.s.pandey@gmail.com

SAMPLE MATRIX — WHAT HARDWARE DOES AND DOESN'T AFFECT

FACTOR	AFFECTED BY CPU/GPU POWER?	NOTE
Which words are likely (the probabilities)	**No**	Determined by the model + your prompt
Speed (tokens per second)	**Yes**	GPUs do the parallel maths far faster
Largest model you can run	**Yes**	Bounded by memory (VRAM/RAM)
Randomness between runs	No — that's the *sampling* settings	See temperature, Section K

COMMON MISCONCEPTION

"A beefier machine makes the model smarter." It doesn't. Intelligence is fixed in the weights. More compute lets you run a *bigger, smarter model*, or the same model faster — the machine never improves a given model's answers. (One caveat: because GPUs add up many numbers in parallel, floating-point rounding can differ by a hair between hardware, so two runs may differ in a rare tie-break — but never in a way you'd notice as "smarter.")

IN ONE SENTENCE

"CPU vs GPU changes the speed and the size of model you can run, not the answer — a slow machine gives the same prediction as a fast one, just later."

C Knowledge Cutoff, Search & the CFO Example

I want to clear up how a model with a fixed cutoff can still cite today's news — and why it sometimes misses it or misjudges it.

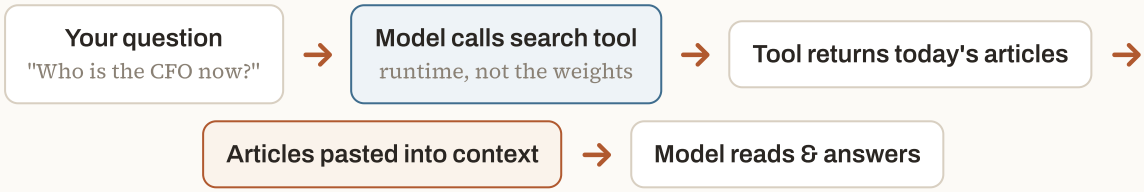
C1 Given a knowledge cutoff, how did the model produce current info (the CFO-resignation example)? Will it answer others correctly after "learning" the CFO resigned — and why didn't it find the news, or find it but miss the severity?

Abhik Dutta · Anonymous · Amit Manchanda

There are two separate stores of knowledge, and untangling them answers all three parts of this question at once:

- **Baked-in knowledge (the weights)** — frozen at the *knowledge cutoff*, the date the training data stops. This is what the model "knows" with no help.
- **Live knowledge (the context)** — anything placed into the prompt *right now*, including results from a **web-search tool**. This is temporary and exists only for this one request.

The model answered with a fresh fact **because it used a search tool:** it issued a query, the tool returned current articles, those articles were pasted into the context, and the model then read them and summarised. The current fact never entered the weights — it rode in through the context and will vanish when the request ends.



Post-cutoff facts arrive through the context, not the weights. Remove the search step and the model can only answer up to its training cutoff.

"Will it answer others correctly now that it 'learned' the CFO resigned?" — No, not from that one chat. Reading a fact in context does not write it back into the weights (recall A2 — inference never changes the model). The next person who asks gets a *fresh* forward pass with none of your context. They'll only get the right answer if the model searches again for them. It didn't "learn" it; it *looked it up*, then forgot.

SAMPLE MATRIX — THE TWO KNOWLEDGE STORES

	BAKED-IN (WEIGHTS)	LIVE (CONTEXT VIA SEARCH)
HOW CURRENT?	Only up to the cutoff date	As current as the search results
PERSISTS ACROSS CHATS?	Yes — same for everyone	No — dies with the request
HELPS THE NEXT USER?	Yes	No — they must search again
FAILURE MODE	Confidently outdated ("hallucinated" current events)	Bad/again missing search results, or misreading them

"Why didn't it find the news — or find it but miss the severity (the fractional-share case)?" These are two different failures, and telling them apart is the whole skill of debugging an AI feature:

Amit Pandey · amit.s.pandey@gmail.com

FAILURE	WHERE IT BROKE	WHAT YOU'D FIX
Didn't find it	*Retrieval* — the search returned nothing relevant (bad query, source not indexed, too recent)	Better query, better/more sources, a news-specific tool
Found it but missed the significance (e.g. that fractional shares implied real distress)	*Reasoning / weighting* — the text was in context but the model didn't connect the detail to its implication	Prompt it to reason about consequences; give domain context; ask for risk analysis explicitly

COMMON MISCONCEPTION

"It read the article, so it understood the stakes." Retrieving text and grasping its significance are separate. A model can faithfully quote "the company issued fractional shares" yet fail to infer "...which signals a distressed reverse split" unless prompted to reason about *why that matters*. Presence in context ≠ correct weighting of importance.

IN ONE SENTENCE

"Post-cutoff facts show up only because a search tool injects them into the current context — the model looks them up, never learns them, so it can still whiff either by not finding the news or by finding it and not appreciating what it means."

D Memory, Statelessness & Conversation History

This is one I really want you to get: the model has no memory, so the "conversation" is an illusion the app keeps up by resending everything.

D1 How does it remember previous discussions? Since LLMs are stateless, does the app really re-send the entire conversation history with every message? Are context windows different on each run?

Anonymous · rohit Chauhan · Anese Syed

Yes — the app really does re-send the whole conversation every turn. The model itself is **stateless**: each request is an isolated function call with *no memory* of any previous call. Between your messages, the model remembers absolutely nothing. What feels like memory is the *application* (ChatGPT, Claude, your own code) keeping the transcript and pasting all of it back in on every new message.

Amit Pandey · amit.s.pandey@gmail.com

ANALOGY

Picture a brilliant consultant with total amnesia every night. To continue yesterday's work, each morning you hand them a folder containing the *entire* conversation so far. They read it in seconds, answer your new question, then forget everything again by evening. The folder — not the consultant — is the memory. The consultant is the (stateless) model; the folder is the context you resend.

WORKED EXAMPLE — WHAT IS ACTUALLY SENT EACH TURN

Watch the payload grow. Each row is one HTTP request to the model:

Turn 1 sends → `[system prompt] + [user: "Hi, I'm building a trading app"]`

Turn 2 sends → `[system] + [user 1] + [assistant 1] + [user: "which model should I use?"]`

Turn 3 sends → `[system] + [user 1] + [assistant 1] + [user 2] + [assistant 2] + [user: "and the cost?"]`

The transcript is resent in full, every time. The model sees turn 3 as one long prompt — it has no independent recollection of turns 1 and 2.

Two consequences fall straight out of this:

- **Cost and latency climb as a chat grows**, because you're re-billing and re-processing the whole history on every turn (see Section M on token charges).
- **Eventually the transcript won't fit** the context window. Apps then truncate old turns, or summarise them into a shorter note, or use a "memory" feature that stores key facts externally and re-injects only those — all just ways of managing what goes back into the context.

"Are context windows different on each run?" The *maximum* is a fixed property of the model (e.g. 128K or 200K tokens). What changes run-to-run is *how much of it you fill*. Turn 1 might use 400 tokens; turn 30 might use 60,000. Same ceiling, different occupancy.

Amit Pandey · amit.s.pandey@gmail.com

SAMPLE MATRIX — MODEL VS APP RESPONSIBILITIES

JOB	WHO DOES IT	HOW
Generate the next answer	The model	One stateless forward pass
Remember the conversation	**The app**	Stores transcript, resends it each turn
Long-term "memory" across chats	**The app**	Saves facts in a database, re-injects relevant ones into context
Handle overflow of the window	**The app**	Truncate / summarise / retrieve

> **IN ONE SENTENCE**
>
> *"The model forgets everything between calls; your app fakes memory by resending the entire transcript each turn, which is exactly why long conversations get slower, pricier, and eventually have to be trimmed."*

E Privacy, Personal Data & Security

Let me walk you through what the model can and can't do with your data — and where I think the real leak risks actually are.

E1 Since we log in, does it track our personal data over time and personalize from it? Can we pass private data to model APIs on cloud? Can one user's conversation (e.g. Aadhaar numbers) leak to another user?

Tota Shailendra Naidu · Sunanda Samaddar · Anonymous · Swostik Padhy

Separate the *model* from the *product* — almost all privacy confusion comes from blurring them.

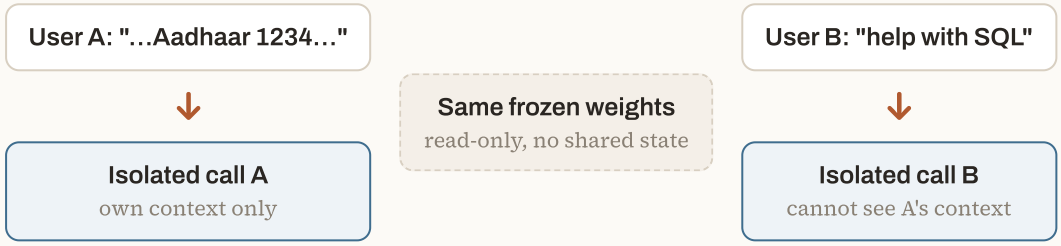
- **The model** does not track you. Inference never updates the weights (A2), so nothing you type is absorbed into the model in real time. It cannot silently accumulate a profile of you on its own.
- **The product around it** can, if designed to. "Personalization" and "memory" features work by the *app* saving facts to a database and re-injecting relevant ones into your context on later chats (D1). That's a storage feature governed by settings and policy — you can usually turn it off — not the model learning you.

Amit Pandey · amit.s.pandey@gmail.com

COMMON MISCONCEPTION

"The model quietly memorises me every time I log in." No single chat changes the model. The only way your data influences a *future* model is if the provider deliberately *trains* a new version on stored logs — a policy/contract decision, not something the running model does. That's exactly the lever you control by choosing the right tier.

Can one user's conversation leak to another in real time? Not through the model. Because every request is stateless and isolated, user B's request is a completely separate forward pass that never sees user A's context. There is no shared live memory between users to leak *from*.



Statelessness = isolation. Two users hitting the same model do not share context. The real leak risks are elsewhere (below).

The *genuine* risk paths — and how they're secured:

REAL RISK	HOW IT COULD HAPPEN	CONTROL
Training on logs	Provider trains a future model on stored chats; a rare "memorised" secret could resurface	Use no-training API/enterprise tiers; don't send secrets on tiers that may train
App-level bugs	Mis-scoped cache/session, over-broad logging, a shared "memory" store	Standard appsec: per-user isolation, access control, log hygiene
Data in transit / at rest	Interception, unencrypted storage	TLS, encryption, region/residency controls

Passing private data to cloud model APIs — yes, safely, if you pick the right arrangement:

SAMPLE MATRIX — DEPLOYMENT TIERS & DATA HANDLING

OPTION	TRAINS ON YOUR DATA?	DATA RESIDENCY / CONTROL	USE WHEN
Consumer app (free/personal)	Sometimes, unless you opt out	Low control	Personal, non-sensitive use
Enterprise / commercial API (with a DPA)	**No** — contractually excluded	Region options, retention limits, SOC2/ISO	Most business use with PII
Cloud-hosted in *your* tenant (e.g. Bedrock/Vertex/Azure)	No	Stays in your cloud account/VPC	Regulated data, strict residency
Self-hosted open-weight model	No — you run it	Total control, never leaves your network	Maximum sensitivity / air-gapped

KEY TAKEAWAY — FOR AADHAAR-GRADE PII

(1) Use a no-training enterprise/self-hosted endpoint. (2) **Redact or tokenise** identifiers before they ever hit the API — replace "Aadhaar 1234 5678 9012" with a placeholder your app can re-map. (3) Encrypt, restrict logs, and isolate storage per user. The model won't hand one user's prompt to another live; you're securing the pipeline around it, not the model's memory.

IN ONE SENTENCE

"The model doesn't track you and can't leak one live user's chat to another because requests are isolated — real privacy work is choosing a no-training tier, redacting PII before it's sent, and locking down your own app's storage and logs."

F Search Engines, SEO & Source Ranking

I'll show you that the LLM doesn't read the whole web — a classic ranking system hands it a shortlist, and it just summarises that.

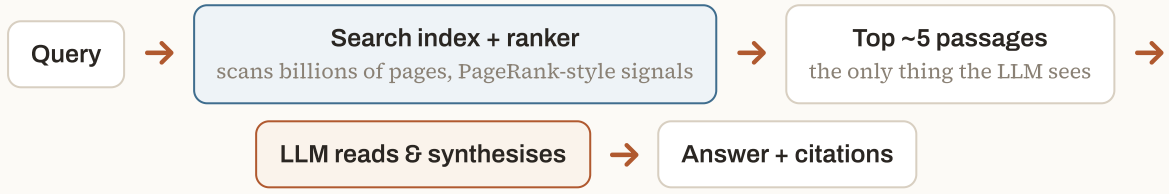
F1 How does an LLM in Google Search rank and recall data across hundreds of pages — and doesn't adding an LLM make it inefficient?

Amit Pandey · amit.s.pandey@gmail.com

How do SEO and site performance affect results, is there PageRank-style authority ranking, and what makes a site LLM-compatible?

Soham Kar | Vinay Chandel | Jishan Baig

The mental model that fixes the "isn't it inefficient?" worry: **the LLM does not rank or read hundreds of pages.** The traditional search engine still does the ranking — fast, cheap, at web scale — and hands the LLM only a tiny shortlist (often the top 3–10 passages). The LLM's job is narrow: *read that shortlist and write a synthesised answer with citations.* This is Retrieval-Augmented Generation (Section L) applied to the web.



Retrieve narrow, then read. The expensive ranking is classic information retrieval; the LLM only ever reads a handful of results, which is what keeps it affordable.

COMMON MISCONCEPTION

"The LLM ingests the whole web per query and ranks it." That would be ruinously expensive. It reads only the retrieved shortlist. Adding the LLM *does* cost more than a blue-links page — which is precisely why engines cap how much text it reads, cache aggressively, and don't run the LLM on every query.

Is there authority-based ranking like PageRank? Yes — at the *retrieval* layer. What gets into the shortlist is still decided by relevance, authority/links, freshness, and quality signals — the descendants of PageRank. The LLM inherits whatever the ranker surfaces; it doesn't re-derive authority from scratch. So "ranking of input sources" lives in the search engine, and the LLM is downstream of it.

SAMPLE MATRIX — WHAT INFLUENCES WHETHER YOUR PAGE IS USED

SIGNAL	AFFECTS RETRIEVAL?	WHY IT MATTERS TO AN ANSWER ENGINE
Authority / inbound links	Strongly	Trusted sources get shortlisted and cited
Relevance to the query	Strongly	Must match the user's intent
Freshness	Yes (esp. news)	Recent facts preferred (Section C)
Crawlability & speed	Yes	If a bot can't fetch/parse it fast, it won't be indexed well
Clear structure & concise facts	Yes	Easy-to-extract passages are easier to quote

Making a site "LLM-compatible" (sometimes called GEO — Generative Engine Optimization): classic SEO still applies for getting *retrieved*; on top of it, optimise for being *easily read and quoted* by a model.

DO THIS	BECAUSE A MODEL…
Use semantic HTML & clear headings; put the answer near the top	…extracts self-contained passages, not whole pages
State facts plainly ("X was founded in 2011")	…quotes crisp, checkable sentences
Add structured data (schema.org), tables, FAQs	…maps structured facts reliably
Ensure server-rendered, fast, crawlable content (not JS-only)	…relies on what the crawler can actually fetch
Be genuinely authoritative & up to date	…(its ranker) favours trusted, fresh sources

> **IN ONE SENTENCE**
>
> *"A search-engine LLM doesn't rank the web — the classic PageRank-style ranker feeds it a short list of top pages and it just summarises those, so you win by being authoritative, crawlable, and written in clear, quotable, well-structured facts."*

G Trust, Reliability & Suitability

Here's how I decide where an LLM's output deserves trust, and where I want a guardrail, a human, or a real calculator in the loop.

Amit Pandey · amit.s.pandey@gmail.com

G1 Can LLMs be trusted to give out-of-the-box solutions for any tech problem?

Prateek Singhal

Trust them the way you'd trust a fast, well-read junior engineer who is *confidently wrong just often enough to matter*. An LLM generates the most *plausible*-sounding solution, which correlates with correctness on common problems but is not the same thing. The more your problem resembles what it saw in training, the more you can lean on it; the more novel, niche, or high-stakes it is, the more you must verify.

> **ANALOGY**
>
> It's autocomplete for solutions. On a paved road (a standard REST API, a common bug) it drives beautifully. Off-road (your proprietary system, a brand-new framework, a subtle concurrency bug) it will still confidently narrate a route — sometimes straight off a cliff — because it's optimised to sound right, not to *be* right.

SAMPLE MATRIX — CALIBRATE YOUR TRUST

SITUATION	TRUST LEVEL	WHY
Boilerplate, common patterns, explaining errors	High (still skim)	Densely represented in training data
Unfamiliar library / your internal system	Medium — verify	Thin or no training coverage; may hallucinate APIs
Security, money, data-destructive ops, compliance	Low — always review & test	Cost of a confident mistake is high
Anything it states as a fact/version/citation	Verify independently	Plausible ≠ true; it can invent specifics

> **IN ONE SENTENCE**
>
> *"Trust an LLM to accelerate and draft, never to be the final authority — it optimises for plausible, so treat every out-of-the-box solution as a smart first draft you still test."*

G2 People use LLMs for emotional/personal decisions — how far has their rationality come?

Anonymous

Modern "reasoning" models are genuinely better at structured, multi-step thinking than models from a couple of years ago — they can lay out options, weigh trade-offs, and catch contradictions. But two limits matter for *personal* decisions specifically:

- **It has no stakes and no real values.** It's simulating what a thoughtful response looks like, not living with the outcome. It doesn't *care* — it patterns-matches caring language.
- **It tends to agree with you (sycophancy).** Trained to be helpful and pleasant, it often validates the framing you gave it, which is the opposite of what a hard personal decision needs.

KEY TAKEAWAY

Useful as a *thinking partner* — to name your options, surface questions, and reflect back your reasoning. Risky as a *decider* for anything emotional, medical, legal, or financial. Ask it to argue the *opposite* of your instinct to counter its agreeableness, and keep a human in the loop for the choice itself.

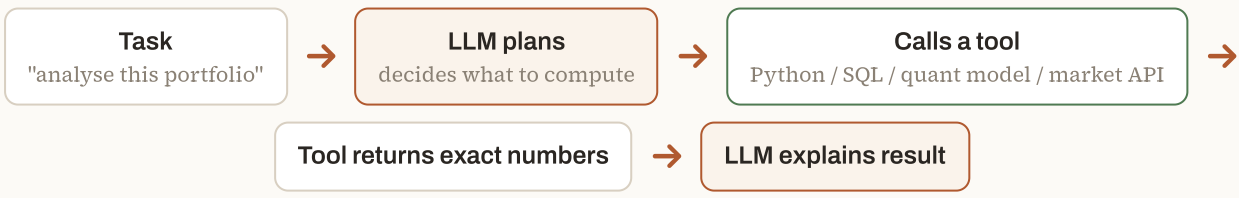
IN ONE SENTENCE

"Its reasoning has come a long way, but it has no stakes and tends to agree with you — great for helping you think, unreliable as the one who decides."

G3 People automate stock trading with LLMs — aren't LLMs bad at numerical data analysis?

Soham Kar

Correct — a bare LLM is genuinely weak at precise numerical work, and for a deep reason: it predicts *tokens*, it doesn't *calculate*. "372 × 48" is answered by what looks right, not by doing the multiplication. So you should **never make the LLM the math engine.** The winning pattern is the opposite: let the LLM *orchestrate* and hand every actual computation to a real tool.



LLM as orchestrator, not calculator. The model chooses *what* to compute; deterministic tools do the arithmetic exactly.

SAMPLE MATRIX — NUMBERS & THE LLM

TASK	BARE LLM	LLM + TOOL
Exact arithmetic / stats	Unreliable	Exact (runs real code)
Reading a chart of numbers into a conclusion	Rough, error-prone	Reliable (compute, then interpret)
Parsing news / earnings text into signals	**Good**	Good
Predicting the market itself	No	Hard even for dedicated quant models — markets are noisy & adversarial

COMMON MISCONCEPTION

"If it automates trading, it must be good at the numbers." The LLM in those systems usually handles *language* (digesting filings, news sentiment, routing decisions) while a separate deterministic engine does the maths and executes trades. And being good at the maths still doesn't make anyone reliably beat the market.

IN ONE SENTENCE

"LLMs are bad at arithmetic because they predict text instead of computing — so use them to read, plan, and explain, and hand every real number to a calculator, code tool, or quant model."

H Architecture

Let me unpack what a Transformer is, whether every model is built the same way, the alternatives, and why we call it a "black box."

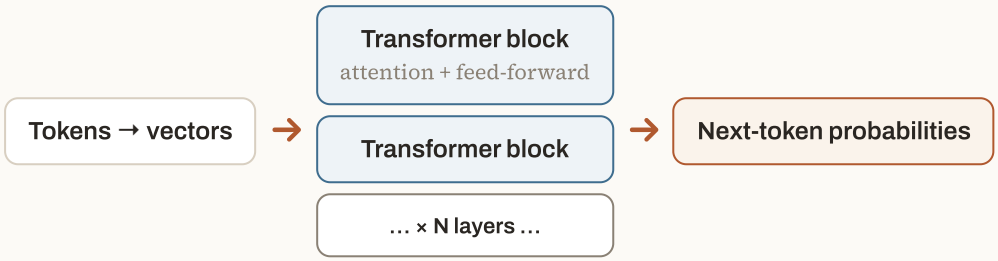
H1 What does "transformer" mean in the context of LLMs?

Ajay Sarswat

"Transformer" is the name of the **neural-network architecture** that essentially all modern LLMs are built on, introduced in the 2017 paper *"Attention Is All You Need."* Nothing to do with electrical transformers or the toys. It's called that because it *transforms* a sequence of token-vectors into a new sequence of vectors, layer after layer, using a mechanism called **self-attention** that lets every token look at every other token *at the same time*.

ANALOGY

Older networks (RNNs) read a sentence one word at a time, like reading through a straw — slow, and prone to forgetting the start by the end. The Transformer spreads the whole sentence on a table and lets every word glance at every other word simultaneously. That parallelism is why Transformers scale to huge data and long context, and why they won.



A Transformer is a tall stack of identical blocks. The vectors pass up through N blocks (12, 32, 80…); the top produces the next-token distribution.

IN ONE SENTENCE

"A Transformer is the 2017 architecture behind today's LLMs — a deep stack of blocks whose key trick, self-attention, lets every token consider every other token in parallel."

H2 Is the Feed-Forward + Multi-Head Attention architecture the same for all LLMs, or can it differ?

Ajay Sarswat

The *skeleton* is shared; the *details* differ a lot. Nearly every LLM stacks the same kind of block — **multi-head attention** (tokens mix information) followed by a **feed-forward network** (each token is processed on its own), wrapped in normalization and residual connections. But within that recipe there are many design choices, and this is exactly where labs differentiate.

SAMPLE MATRIX — THE SAME SKELETON, DIFFERENT PARTS

COMPONENT	COMMON VARIATIONS
Attention	Multi-Head (MHA) → Grouped-Query (GQA) / Multi-Query (MQA) to save memory; sliding-window; sparse
Position info	Learned positions → RoPE (rotary) → ALiBi
Normalization	LayerNorm → RMSNorm; post-norm → pre-norm
Feed-forward activation	ReLU → GELU → SwiGLU
Dense vs sparse	Every block active (dense) → Mixture-of-Experts (only a few "expert" sub-networks fire per token)
Depth / width	Number of layers, heads, and hidden size all vary by model

KEY TAKEAWAY

Think "same genre, different arrangements." If you can read one Transformer diagram you can read them all — but don't assume two models are identical inside. Choices like GQA and MoE materially change speed, memory, and cost.

IN ONE SENTENCE

"Every LLM uses the attention-plus-feed-forward block, but the knobs — attention variant, position encoding, activation, dense vs mixture-of-experts — differ from model to model."

H3 Is there any model other than the Transformer? Isn't it very resource-heavy (all those K/V pairs per word) — is there a better architecture?

Abinash Gupta · Soham Kar

Yes to both. Transformers *are* memory-hungry: full attention compares every token with every other (next question), and generation keeps a growing **KV cache** — the stored Key/Value vectors for every previous token — which balloons with long context. That has driven two responses: *efficiency tricks inside Transformers*, and *alternative architectures*.

SAMPLE MATRIX — ARCHITECTURES & ALTERNATIVES

ARCHITECTURE	SCALING WITH SEQUENCE LENGTH	STATUS
RNN / LSTM (pre-2017)	Linear, but sequential & forgetful	Largely superseded for LLMs
Transformer (full attention)	Quadratic — $O(n^2)$	Dominant; best quality
State-Space Models (Mamba/S4), RWKV, linear attention	~Linear — O(n), small fixed memory	Promising for very long context; catching up on quality
Hybrids (e.g. Jamba)	Mixed	Combine attention + SSM to balance quality & efficiency

And efficiency tricks that keep Transformers practical without changing the core: **GQA/MQA** (shrink the KV cache), **FlashAttention** (compute attention with far less memory traffic), **sliding-window / sparse attention** (don't attend to everything), and **KV-cache quantization**.

COMMON MISCONCEPTION

"Something must have replaced the Transformer by now." Not for frontier quality — as of today Transformers (with efficiency tricks) still lead. The alternatives are real and improving fast, especially for long-context and on-device use, but there's no clean, across-the-board replacement yet.

IN ONE SENTENCE

"Transformers are expensive because attention is quadratic and the KV cache grows with length — there are lighter alternatives (Mamba, RWKV, linear attention) and many efficiency tricks, but nothing has yet dethroned the Transformer at the top."

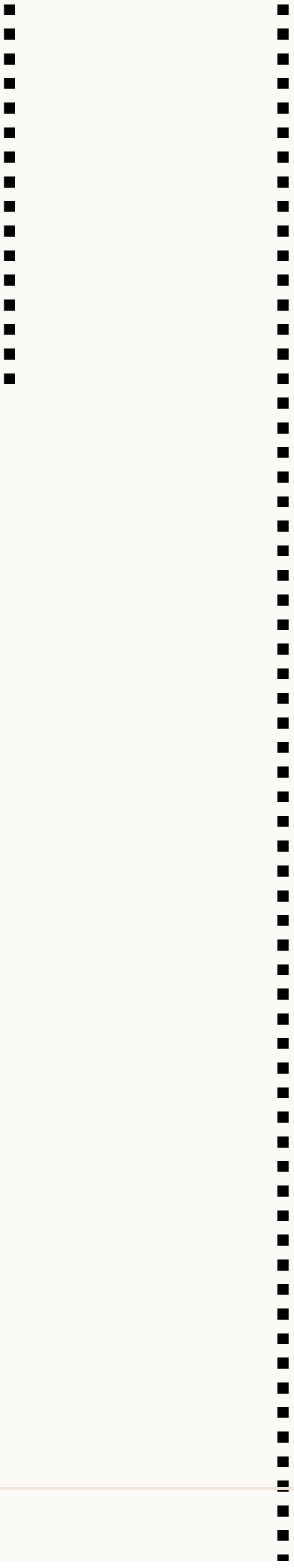
H4 Is the $O(n^2)$ complexity common to all LLMs or specific ones?

Prateek Singhal

$O(n^2)$ is a property of **full self-attention** specifically, not a law that binds every LLM. It arises because each of the n tokens computes a relationship with all n tokens → $n \times n$ comparisons. Double the input, quadruple the attention work. Models that use full attention (most classic Transformers) inherit it; models that use linear-attention or state-space designs do *not*.

```
4 tokens → 16 comparisons    8 tokens → 64 comparisons
```


Amit Pandey · amit.s.pandey@gmail.com



Amit Pandey · amit.s.pandey@gmail.com

The attention matrix is n × n. Doubling tokens from 4 to 8 quadruples the cells — that's the $O(n^2)$ cost that long prompts pay.

IN ONE SENTENCE

"$O(n^2)$ comes from full attention comparing every token with every token, so it applies to standard Transformers but not to linear-attention or state-space models that are built to scale linearly."

H5 Do LLMs use any specific probabilistic algorithm?

Jayakrishna Araveti

Not a classical named one (no Naive Bayes, no HMM). An LLM is a neural network trained to model one probability: `P(next token | all previous tokens)`. A whole sentence's probability is just these multiplied together — this is the **autoregressive** factorization:

> *P("the cat sat") = P("the") × P("cat" | "the") × P("sat" | "the cat")*

The pieces that *are* named are: **softmax** (turns the network's raw scores into a probability distribution — Section J), and the **decoding/sampling strategy** that picks a token from that distribution (greedy, temperature, top-k, top-p — Section K). Training uses **gradient descent** to minimise prediction error. So it's "a learned distribution + a sampling rule," not an off-the-shelf probabilistic algorithm.

IN ONE SENTENCE

"It isn't running a textbook probabilistic algorithm — it's a neural net that learns P(next token | context), turns scores into probabilities with softmax, and then samples from them."

H6 What is a "black box," why is it used in LLMs, and how is it integrated?

Abinash Gupta

"Black box" isn't a component you add — it's a *description* of the situation: we can see the inputs and outputs, but we cannot fully explain *why* a particular arrangement of billions of weights produced a particular answer. There are no human-written rules inside to read; the behaviour is *learned* and spread across the network as distributed patterns, so no single weight means "the capital of France."

> **ANALOGY**
> We understand a single neuron the way we understand a single transistor, and we can watch the whole thing run — yet we can't point to where a "thought" lives, just as neuroscience can't point to the exact neurons holding your memory of breakfast. Understanding the parts doesn't hand you the whole.

Why "used"? It's the price of learned systems: they solve problems we can't write explicit rules for, but in exchange we lose a clean, step-by-step explanation. **How it's integrated:** because you can't read its internals, you engineer *around* it — evaluations and test suites, guardrails and validators on the output, logging, and human review for high-stakes calls. A growing field, *mechanistic interpretability*, is slowly prying the box open (probing neurons, visualising attention), but in production you treat the model as a component you *measure* rather than one you *read*.

> **IN ONE SENTENCE**
> *"'Black box' just means we see an LLM's inputs and outputs but can't fully explain the billions-of-weights 'why' — so we wrap it in tests, guardrails, and human review instead of reading its internals."*

Embeddings, Dimensions, Vectors & the Tokenizer

I'll take you from a single word to a list of numbers — the concrete pipeline, walked end to end.

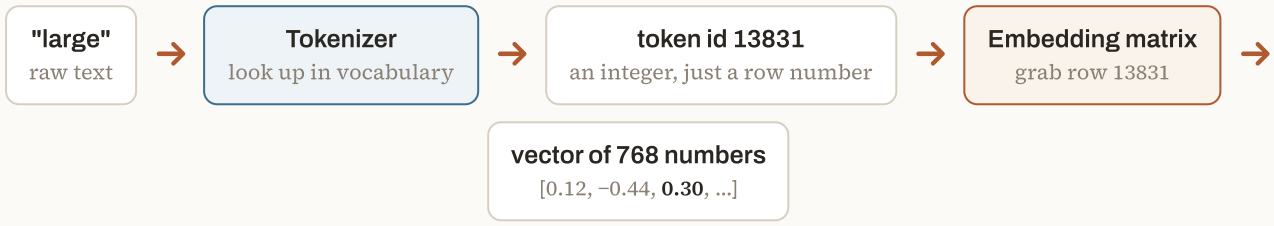
I1 What do "dimensions" (512, 768) mean? How is a word turned into an embedding, how is the embedding matrix built, how do we pick the dimension — and can you walk the concrete example (the word "large", id 13831, the value .30)?

Prateek Singhal · Akshay Patil · Satish Hirwmath · Anonymous · Jishan Baig

An **embedding** is a list of numbers (a vector) that represents a token's meaning. The **dimension** is simply *how many numbers are in that list*. "768 dimensions" means every token is represented by 768 numbers. Each number is a coordinate along one learned axis of meaning — the axes aren't human-labelled ("royalty", "size"); they're whatever the model found useful during training.

> **ANALOGY**
>
> To locate a place on Earth you need 2 numbers (lat, long) — 2 dimensions. To describe a colour you need 3 (R, G, B). Word meaning is far richer, so we use hundreds of numbers. More dimensions = more room to place similar words near each other and different words far apart.

The pipeline, end to end, for the word "large":



Word → id → vector. "large" becomes the integer 13831 (its slot in the vocabulary), and 13831 is used to look up one row of the embedding matrix. That row is 768 numbers; the **0.30** is just the value on one of those 768 axes — no single meaning on its own.

How the embedding matrix is built. It's one big lookup table with shape `[vocabulary size × dimension]` — e.g. `[50000 × 768]`. One row per possible token. It starts as *random* numbers and is **learned during training** like every other parameter (Section A): each row drifts so that tokens used in similar ways end up with similar vectors. So the tokenizer decides *which row*; training decides *what's in the row*.

SAMPLE MATRIX — A TINY EMBEDDING TABLE (TOY: 4 DIMENSIONS INSTEAD OF 768)

TOKEN (ID)	DIM 1	DIM 2	DIM 3	DIM 4
"king" (5921)	0.21	0.85	−0.10	0.30
"queen" (7833)	0.19	0.83	−0.08	0.33
"large" (13831)	0.12	−0.44	0.30	0.05
"apple" (1204)	−0.60	0.05	0.72	−0.11

Notice "king" and "queen" have *nearly identical* vectors (similar meaning), while "apple" is far away. That closeness is the entire point of embeddings — meaning becomes distance. Your **.30** is "large" on dim 3 here; on its own it's meaningless, it only matters as part of the whole coordinate.

How is the correct dimension chosen? There's no formula — it's a *hyperparameter* the model designers tune, trading expressiveness against cost. It must equal the model's hidden size and stay fixed for that model. Rough guide: 384–768 for compact embedding models, 768–1024

Amit Pandey · amit.s.pandey@gmail.com

mid-size, 4096+ inside large LLMs. Bigger captures finer distinctions but costs more memory and compute.

> **IN ONE SENTENCE**
> *"'768 dimensions' means each token is a list of 768 learned numbers; the tokenizer turns 'large' into the id 13831, that id picks row 13831 of the embedding table, and the 0.30 is just one coordinate of that 768-number vector."*

I2 Tokenizer specifics — is it fixed for code too or different? Does each model have its own tokenizer + embeddings? When you upload a file, is each word a token? Does understanding BPE help get better results / less hallucination?

Soham Kar · Anonymous · Anese Syed · Jayakrishna Araveti

Most LLMs use **BPE (Byte-Pair Encoding)** or a close cousin. It works on *bytes*, so a single tokenizer handles English, other languages, emoji, and code alike — there isn't a separate "code tokenizer." But the *result* differs by content: code has lots of symbols, indentation, and rare identifiers, so it often splits into more tokens per character than plain prose.

> **COMMON MISCONCEPTION**
> "One word = one token." Not reliably. Common words are one token; rare or long words split into sub-word pieces; spaces, punctuation, and case all count. As a rule of thumb, English averages **~4 characters per token** (~¾ of a word). When you upload a file, it's tokenized the same way — not one-token-per-word.

SAMPLE MATRIX — HOW TEXT SPLITS INTO TOKENS

INPUT	TOKENS (ILLUSTRATIVE)	COUNT
`cat`	`["cat"]`	1
`unbelievable`	`["un","believ","able"]`	3
`ChatGPT`	`["Chat","GPT"]`	2
`2025-11`	`["202","5","-","11"]`	4
`def add(a,b):`	`["def"," add","(","a",",","b","):"]`	7

Does each model have its own tokenizer + embeddings? Yes. Each model family ships its own tokenizer (GPT's tiktoken, Llama's SentencePiece, etc.) *and* its own trained embedding matrix. A token id from one model is meaningless to another — id 13831 is "large" only for the tokenizer that assigned it. Tokenizer and embeddings are a matched pair, trained and used together.

DOES KNOWING BPE HELP?

Yes, in practical ways — though not directly as a hallucination cure. It helps you (1) *estimate cost and context use* (billing is per token, Section M); (2) understand why odd spacing, typos, long numbers, or exotic characters cost extra tokens and sometimes trip the model; (3) know that number- and character-level tasks are shaky *because* digits get chopped into odd pieces. It sharpens your intuition and trims cost; hallucination is better fixed with grounding (RAG), verification, and prompting.

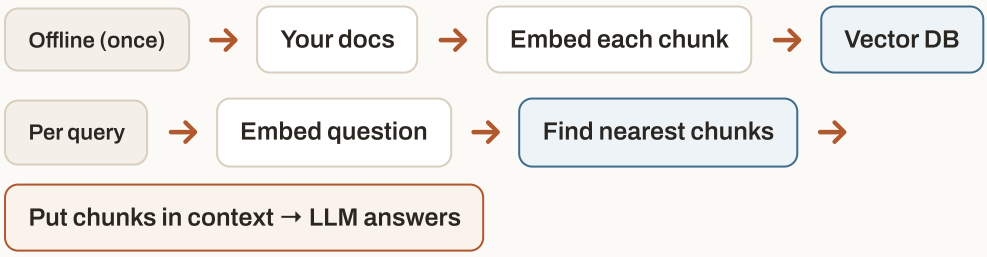
IN ONE SENTENCE

"One BPE tokenizer handles prose and code by working on bytes, words are often split into sub-word pieces (so 'each word = a token' is false), every model has its own matched tokenizer + embeddings, and knowing BPE mainly helps you predict cost and quirks — not directly reduce hallucination."

I3 How does a vector database relate to / get used in an LLM, and is vectorization mandatory for RAG?

Ajay Sarswat · Anonymous

A **vector database** is a *separate* system from the LLM. It stores embeddings of *your* documents and lets you search them by meaning: give it a query vector and it returns the closest document vectors (nearest-neighbour search). It's the "find the relevant passages" half of RAG (Section L); the LLM is the "read and answer" half.



The vector DB feeds the LLM. It never lives "inside" the model; it's an external retrieval store whose results get pasted into the context.

Is vectorization mandatory for RAG? No. RAG just means "retrieve relevant text, then generate an answer from it." *How* you retrieve is open:

RETRIEVAL METHOD	NEEDS VECTORS?	GOOD WHEN
Vector / semantic search	Yes	Meaning matters, wording varies ("car" ≈ "automobile")
Keyword search (BM25)	No	Exact terms, codes, names matter
SQL / database query, API call	No	Structured data, live systems
Hybrid (keyword + vector)	Partly	Best of both — common in production

IN ONE SENTENCE

"A vector database is an external store that finds meaning-similar chunks to feed into the LLM's context — it's the retrieval half of RAG, and it's popular but not mandatory, since keyword, SQL, or hybrid search can retrieve just as well."

J Attention (Q / K / V) & Softmax

Here I'll explain the mechanism that lets tokens read each other — and why that "softmax" step has such an odd name.

J1 Are Q, K, V generated during training or inference, and where do persona and context sit in the attention formula?

Ashutosh Srivastava · Jishan Baig

Split it into two things that are easy to conflate:

- The **weight matrices** W_Q, W_K, W_V that *produce* Q, K, V are **learned during training** — they're part of the model's parameters and then frozen.
- The **actual Q, K, V vectors** are computed **fresh on every forward pass** — so yes, at inference too. Each token's embedding is multiplied by those frozen matrices to get that token's Query, Key, and Value.

ANALOGY — A ROOM OF PEOPLE NETWORKING

Everyone in the room (each token) wears a **name-tag = Key** ("I'm about rivers"), silently forms a **Query** ("I'm looking for water-related context"), and can hand over a note of **Value** (the actual info they carry). To update yourself, you compare *your* Query to *everyone's* Key, listen most to whoever matches, and collect a blend of their Values weighted by how well they matched.

Mechanically, that comparison-and-blend is exactly the attention formula. (Your note wrote it as "K·Q^T·V"; the standard form is the same idea in this order:)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$

- $Q \cdot K^T$ — every token's Query dotted with every token's Key → a grid of match scores.
- $\div \sqrt{d_k}$ — scale down so the numbers don't blow up (keeps training stable).
- **softmax(...)** — turn those scores into attention weights that sum to 1 (Section J2).
- **... · V** — take a weighted blend of the Values using those weights. That blend is the token's new, context-aware vector.

WORKED EXAMPLE — "I SAT BY THE RIVER BANK"

Focus on the token `bank`. Its Query is compared with the Keys of the earlier words to get raw scores, softmax turns those into weights, and the output is that blend of Values:

SAMPLE MATRIX — ATTENTION FOR THE WORD "BANK"

ATTENDS TO	Q·K SCORE	AFTER SOFTMAX (WEIGHT)
"the"	1.0	0.04
"river"	**4.0**	**0.84**
"bank" (itself)	2.0	0.12

Because "river" scores highest, its Value dominates the blend, so `bank`'s new vector leans toward the *waterside* meaning rather than the *financial* one — that's attention "understanding context" (Section B) in action.

Where do persona and context sit in this? They aren't a separate term in the formula — they're just *more tokens in the input*. The system prompt ("You are a terse legal assistant") and the conversation history each get their own Q, K, and V rows like any other token. They shape the

output by being *Keys* that later tokens match against and *Values* that get mixed in. Instructions tend to earn high attention weights precisely because the answer tokens "query" for them.

SAMPLE MATRIX — WHAT Q, K, V EACH MEAN

VECTOR	THE QUESTION IT ANSWERS	NETWORKING ANALOGY
Query (Q)	"What am I looking for right now?"	Who I want to hear from
Key (K)	"What am I about / what do I offer?"	My name-tag
Value (V)	"The actual information I pass on if attended to"	What I say when picked

The $W_Q/W_K/W_V$ matrices are learned in training and frozen; the Q/K/V vectors above are recomputed on every forward pass, including at inference.

IN ONE SENTENCE

"The matrices that make Q, K, V are learned once in training; the Q, K, V vectors themselves are recomputed every forward pass — and persona and context aren't special terms, they're just extra tokens whose Keys and Values get attended to."

J2 Why is it named "softmax"?

Shashwat Kotiyal

Because it's a **"soft" (smooth) version of the "max"** operation. A hard max (argmax) picks the single biggest value and gives it *all* the weight — winner-takes-all, `[1, 0, 0]`. Softmax instead spreads the weight into a smooth probability distribution: the biggest score still gets the most, but the others keep a nonzero share, and everything sums to 1.

WORKED EXAMPLE

Take three raw scores `[2.0, 1.0, 0.1]`. Exponentiate each (e^x), then divide by the total so they sum to 1:

SCORE	E^{SCORE}	SOFTMAX (÷ SUM)	HARD MAX WOULD GIVE
2.0	7.39	**0.66**	1
1.0	2.72	0.24	0
0.1	1.11	0.10	0
sum	*11.22*	*1.00*	1

Two reasons it's used everywhere in LLMs: it produces valid **probabilities** (for the next token, and for attention weights), and unlike hard max it's **differentiable** — smooth enough to train with gradient descent. A hard "pick the max" has no gradient to learn from.

> **IN ONE SENTENCE**
>
> *"It's a soft, smooth stand-in for 'take the maximum' — instead of giving the top score everything, it turns all the scores into probabilities that sum to 1, which is both meaningful and trainable."*

K Sampling Parameters — Temperature, Top-p / Top-k

Let me demystify the dials that control randomness — how they work, and why they're not the same as telling the model to "be creative."

K1 If higher temperature flattens the probabilities, how is a correct decision still made? Does temperature interfere with the embedded values? And can it be controlled from the prompt ("be creative") instead of the API?

Akshay Patil · Ajay Sarswat · Jishan Baig

Temperature is a single number applied *right at the end*, during sampling. It divides the scores (logits) before softmax: **low temperature sharpens** the distribution (the top token dominates → near-deterministic), **high temperature flattens** it (lower-ranked tokens get a realistic chance → more variety).

> **WORKED EXAMPLE — SAME THREE CANDIDATES AT THREE TEMPERATURES**
>
> Logits: `cat 2.0`, `dog 1.0`, `banana 0.1`. Watch the probabilities change while the *ranking* does not:

SAMPLE MATRIX — TEMPERATURE RESHAPES THE ODDS

TOKEN	T = 0.5 (SHARP)	T = 1.0 (NORMAL)	T = 1.5 (FLAT)
cat	**0.87**	**0.66**	**0.52**
dog	0.12	0.24	0.30
banana	0.01	0.10	0.18

So how is a correct decision still made at high temperature? Because flattening doesn't change *which* token is most likely — "cat" still wins at every setting. High temperature only raises the *chance* of occasionally picking a lower-ranked token; it doesn't make the wrong one the favourite.

Amit Pandey · amit.s.pandey@gmail.com

That's why factual/coding tasks use low temperature (stay on the top choice) and brainstorming uses high (allow surprises).

> **DOES IT INTERFERE WITH THE EMBEDDED VALUES?**
> No. Temperature never touches the embeddings or the weights. It only rescales the *final* logits at sampling time — a knob on the output distribution, applied after all the model's real computation is done. Embeddings are untouched.

Can you set it from the prompt ("be creative") instead of the API? These are two different mechanisms, and conflating them is a classic mistake:

	SAYING "BE CREATIVE" IN THE PROMPT	SETTING `TEMPERATURE` IN THE API
What it changes	The *content* the model aims for — it shifts the probability distribution itself (via context)	The *randomness of sampling* from whatever distribution exists
Where it acts	Inside the model (it's just more context)	Outside, after the model, at the sampling step
Precise numeric control?	No — it's a vibe, not a value	Yes — an exact dial (e.g. 0.2 vs 1.3)

You genuinely cannot set the number 1.3 by typing prose; the API parameter is the real control. But wording still matters — "be creative" nudges the distribution toward varied words. In practice you combine them: prompt for the *style*, set temperature for the *degree of chance*.

> **IN ONE SENTENCE**
> *"Temperature is an after-the-fact dial that flattens or sharpens the output odds without touching embeddings, the top token usually still wins so answers stay sane, and it's a different lever from telling the model 'be creative' — one controls sampling randomness, the other nudges the content."*

K2 Difference between Top-p and Top-k, with an example.

Biswajit Bers

Both trim the candidate list *before* sampling, so the model can't pick absurd low-probability tokens — but they trim differently:

- **Top-k** — keep the k highest-probability tokens, throw away the rest, renormalize, then sample. Fixed *count*.
- **Top-p (nucleus)** — keep the smallest set of tokens whose probabilities *add up to at least p*, then sample. Adaptive count — it shrinks when the model is confident and grows when it's unsure.

WORKED EXAMPLE

Next-token probabilities: `red 0.50`, `blue 0.25`, `green 0.15`, `pink 0.06`, `teal 0.04`.

SAMPLE MATRIX — SAME DISTRIBUTION, TWO STRATEGIES

STRATEGY	KEEPS	WHY
Top-k = 2	red, blue	The 2 highest, always exactly 2
Top-p = 0.90	red, blue, green	0.50 + 0.25 + 0.15 = 0.90 reaches the threshold; pink & teal dropped

The key contrast is what happens when confidence changes. If the model is *very* sure (`red 0.95`, rest tiny), **Top-p = 0.9** keeps just `red` (it alone clears 0.9), while **Top-k = 2** still drags in a barely-relevant `blue`. Top-p adapts to certainty; top-k doesn't. That adaptiveness is why top-p is the more common default.

IN ONE SENTENCE

"Top-k keeps a fixed number of the best tokens; top-p keeps just enough of the best to cover a probability threshold — so top-p automatically narrows when the model is confident and widens when it isn't."

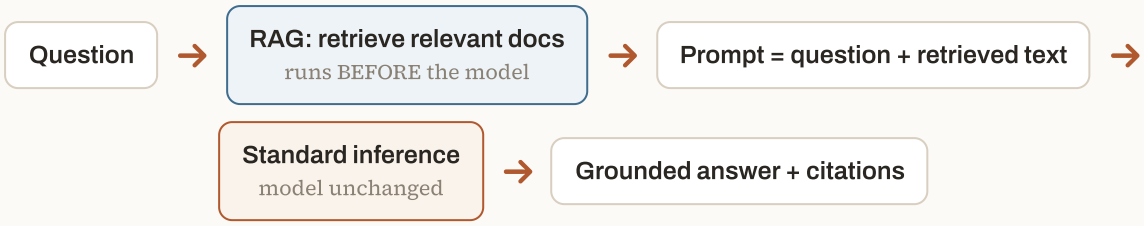
L RAG & Fine-Tuning

I'll lay out the two ways I make a model "know" your stuff — one changes the prompt, the other changes the weights.

L1 Where is the RAG layer placed (before or after prediction)? Can RAG be added on top of a frontier model? How do you set context in deep research and improve RAG — and could the court-cases use case be done via RAG (like NotebookLM) instead of chat?

Ajay Sarswat · Snehith Allamraju · Jayakrishna Araveti · Ankit Sahu

RAG sits *before* prediction. It's a retrieval step that runs first, finds relevant text, and stuffs it into the prompt; *then* the model does completely ordinary inference on that enriched prompt. It's in front of the model, never inside or after it.



RAG is prompt construction, not a model change. Because it only edits the input, it bolts onto *any* model — including a frontier model — with zero retraining.

Can it be added on top of standard inference for a frontier model? Yes — that's the whole appeal. RAG is just "search, then paste results into the context," so it works on top of any hosted model as-is.

Setting context in "deep research," and improving RAG. Deep research is *agentic* RAG: the model searches, reads, searches again, and synthesises across many rounds. You set its context by scoping the question, providing seed sources, and constraining what's authoritative. To improve any RAG system:

- **Chunk well** — split documents into passages that are self-contained (not mid-sentence).
- **Retrieve better** — hybrid (keyword + vector), then a *reranker* to put the best passage first.
- **Keep context clean** — a few highly relevant chunks beat many noisy ones (see M1).
- **Rewrite the query** and **demand citations** so answers stay grounded and checkable.

> **WORKED EXAMPLE — COURT CASES AS RAG (THE NOTEBOOKLM PATTERN)**
>
> Yes, and it's the *better* design than open chat. Setup: (1) upload the case files & judgments; (2) chunk and index them in a retrieval store; (3) at query time, retrieve only passages from *your* documents; (4) instruct the model to answer **only** from retrieved text and cite the source paragraph. Result: answers grounded in the actual filings, every claim traceable to a document — exactly what NotebookLM does. Plain chat (no RAG) would answer from generic training memory and is far likelier to hallucinate a case.

Amit Pandey · amit.s.pandey@gmail.com

SAMPLE MATRIX — THREE WAYS TO GIVE A MODEL YOUR KNOWLEDGE

	RAG	FINE-TUNING	LONG CONTEXT (PASTE IT ALL)
Changes the model?	No	Yes (weights)	No
Freshness	Live — update the store anytime	Stale until retrained	Live
Best for	Facts, documents, citations	Style, format, a skill	One-off, small sources
Cost model	Retrieval infra + tokens	Training run up front	Big per-request token cost

IN ONE SENTENCE

"RAG runs before the model as a retrieval step that enriches the prompt, so it bolts onto any frontier model with no retraining — and grounding a court-cases assistant in your own filings with citations (the NotebookLM pattern) is exactly what it's for."

L2 When we fine-tune or use RAG, are we essentially just updating the embeddings of particular words — is that the right mental model?

Soham Kar

No — that's the mental model to drop. Neither one is "editing a few word vectors."

- **RAG changes nothing inside the model.** No weights, no embeddings. It only changes the *input* by adding retrieved text to the context. The model is byte-for-byte identical before and after.
- **Fine-tuning updates the weights** — potentially across the whole network — via more gradient-descent training on new examples. It reshapes *behaviour* (tone, format, a narrow skill), not "the embedding of the word *invoice*." Yes, embedding rows are among the parameters that can shift, but the effect and intent are network-wide, not a targeted dictionary edit.

ANALOGY

RAG = handing an expert a briefing folder before they answer (they're unchanged; you changed what's on their desk). **Fine-tuning** = sending that expert back to school so their instincts change permanently. Neither is "rewriting a few definitions in their dictionary."

	WHAT ACTUALLY CHANGES	PERMANENT?
RAG	The prompt (external context)	No — per request
Fine-tuning	The model's weights (broadly)	Yes — bakes into a new model
"Updating word embeddings"	Not what either one does	—

IN ONE SENTENCE

"RAG edits the prompt and leaves the model untouched; fine-tuning retrains the weights broadly to change behaviour — neither is simply updating the embeddings of particular words."

M Prompting, Context Strategy & Tokens

Here's how I feed the model, how I spend fewer tokens, and how billing actually works.

M1 Is it better to give a large context in one prompt, in parallel, or one-by-one?

Ajay Sarswat

It depends on whether the pieces need to *see each other*. And beware a real effect called **"lost in the middle"**: as a context grows very large, the model reliably attends to the start and end but gets fuzzier on the middle — so *more context is not automatically better*.

SAMPLE MATRIX — CHOOSE BY TASK SHAPE

APPROACH	BEST WHEN	WATCH OUT FOR
One big prompt	The task needs cross-referencing across all the material at once	Cost, and "lost in the middle" if it's huge
One-by-one (sequential)	Steps depend on each other; each answer feeds the next	Slower; you manage the hand-offs
In parallel	Sub-tasks are independent (summarise 20 files separately)	No cross-talk between the parallel calls

KEY TAKEAWAY

Give only the *relevant* context, not everything you have. Combine when the model must connect the dots; split (parallel) when the chunks are independent; go sequential when later steps genuinely depend on earlier answers.

Amit Pandey · amit.s.pandey@gmail.com

IN ONE SENTENCE

"Put it in one prompt when the parts must talk to each other, run parallel calls when they're independent, go step-by-step when each builds on the last — and don't over-stuff, because giant contexts lose detail in the middle."

M2 Can we design ways to reduce tokens by working around the embeddings + positional math?

Jayakrishna Araveti

You can't *bypass* the tokenizer/embedding/position machinery — every input becomes tokens, full stop. But you don't need to touch the maths to cut tokens; you just **send less text**. The savings all come from what you put in the context, not from tricking the internals.

SAMPLE MATRIX — REAL TOKEN-REDUCTION TACTICS

TACTIC	HOW IT SAVES TOKENS
Retrieve, don't paste (RAG)	Send only the relevant chunks, not whole documents
Summarise old history	Replace 20 turns with a short recap (Section D)
Prompt caching	Reuse an already-processed prefix (system prompt, docs) so you're not re-billed to re-read it
Concise prompts, no redundancy	Fewer input tokens directly
Cap the output	Output is often the pricier side (M5)

IN ONE SENTENCE

"You can't shortcut the embedding/position maths, but you rarely need to — cut tokens by sending less (retrieval, summaries, caching, brevity), which is where the real savings live."

M3 Can we use Skills to save tokens?

Aman Anand

Yes — that's largely their point. A "skill" is a reusable capability (instructions/tools) that's loaded **on demand** rather than crammed into every prompt. Instead of carrying the full instructions for ten possible tasks in every request, you invoke only the one relevant skill when it's needed.

TWO WAYS SKILLS SAVE TOKENS

(1) **Load-on-demand**: only the relevant skill's prompt enters context, not all of them. (2) **Offloading**: a skill often calls deterministic tools/code, so the model generates fewer tokens doing work by hand. Caveat — a loaded skill's own text still costs tokens; the win is loading *only what's relevant* and reusing it, versus a bloated all-in-one prompt.

IN ONE SENTENCE

"Skills save tokens by loading only the capability you need for a task instead of stuffing every instruction into every prompt — and by handing work to tools so the model writes less."

M4 Which format is best for a system prompt (markdown / JSON / YAML) to optimize cost / reduce token usage?

Ramasamy A · Shashank Pandey

Markdown is the usual sweet spot for *instructions*: it's readable, models are heavily trained on it, and it carries little structural overhead. JSON and YAML add punctuation tokens (braces, quotes, colons, indentation) that cost more without helping the model follow instructions — reserve them for when you truly need machine-parseable structure.

WORKED EXAMPLE — THE SAME INSTRUCTION, THREE FORMATS

Roughly, structural characters become extra tokens. For the same content:

FORMAT	LOOKS LIKE	RELATIVE TOKEN OVERHEAD
Markdown	`## Tone\nConcise, formal`	Lowest
YAML	`tone: Concise, formal`	Low–medium
JSON	`{"tone": "Concise, formal"}`	Highest (quotes + braces)

KEY TAKEAWAY

Write the *system prompt* in Markdown to save tokens and maximise instruction-following. Use JSON/YAML only for data you or the model must parse programmatically (and for the model's *output* when your code consumes it — M6).

> **IN ONE SENTENCE**
> *"Use Markdown for system prompts — it's cheapest in tokens and best-understood; JSON and YAML mostly add structural punctuation you pay for and the model doesn't need to follow instructions."*

M5 Are token charges based on input, output, or both — and how do we control response length? Does asking for reasoning consume extra tokens even when we didn't request it?

Anonymous · Prateek Singhal

Both are billed — and typically *output tokens cost more per token than input tokens*. Everything you send (system prompt + full history + your message) is input; everything the model generates is output.

SAMPLE MATRIX — WHAT YOU PAY FOR

TOKEN TYPE	BILLED?	NOTE
Input (prompt + history + docs)	Yes	Grows every turn (Section D)
Output (the answer)	Yes — usually the pricier rate	Control with `max_tokens` and by asking for brevity
Reasoning / "thinking" tokens	Yes, on reasoning models	Generated and billed even though you may not see them

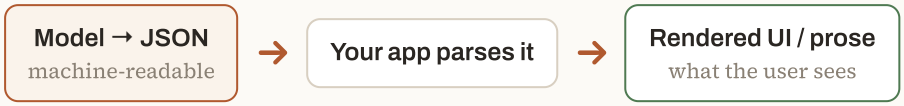
Controlling length: set `max_tokens` (a hard cap), and ask explicitly ("answer in 2 sentences", "bullet list, no preamble"). **Reasoning tokens:** yes — on a reasoning model, the hidden chain-of-thought is real generated output and is billed, even if the UI hides it and you didn't ask for it. If you don't want to pay for that, use a non-reasoning model or turn the reasoning effort down where the API allows.

> **IN ONE SENTENCE**
> *"You pay for both input and output (output usually dearer), you cap length with max_tokens plus explicit 'be brief' instructions, and reasoning models bill you for hidden thinking tokens whether or not you asked to see them."*

M6 If we show the LLM's output directly to the user, JSON output doesn't make sense — it's only for further technical processing. Correct?

Anushka Sehrawat

Correct in spirit. JSON is a *data contract* for **your code**, not a reading format for humans. You'd never show a user raw `{"status":"ok","items":[…]}`. The right flow: ask the model for JSON, let your app *parse* it, then render a nice human UI from the parsed data.



JSON is an intermediate step, even in user-facing apps. For direct-to-human answers with no code in between, ask for Markdown/prose instead.

IN ONE SENTENCE

"Right — JSON is for your code to parse, not for humans to read; use it as a data contract you then render, and ask for Markdown/prose when the text goes straight to the user."

M7 In Cursor's Plan mode, the plan.md can get large — how is it handled when the file is too big?

Shubh Mandalia

The same context-window limit applies (Section D): a file only "exists" for the model when its text is in the context, and the window is finite. When `plan.md` gets too big to send whole, tools handle it the way any large-context system does — they don't blindly send all of it every turn:

- **Summarise / compact** — replace older or verbose sections with a condensed version.
- **Retrieve relevant slices** — pull only the sections related to the current step (RAG over the file).
- **Chunk & reference** — work section-by-section instead of all at once.

PRACTICAL TIP

Keep plans modular and pruned — short, current, well-structured plans stay fully in context and get better attention than one sprawling document that has to be summarised or truncated.

IN ONE SENTENCE

"A too-big plan.md can't all fit the context window, so the tool summarises, retrieves only the relevant sections, or works in chunks — which is why keeping the plan lean gives you better, cheaper results."

Amit Pandey · amit.s.pandey@gmail.com

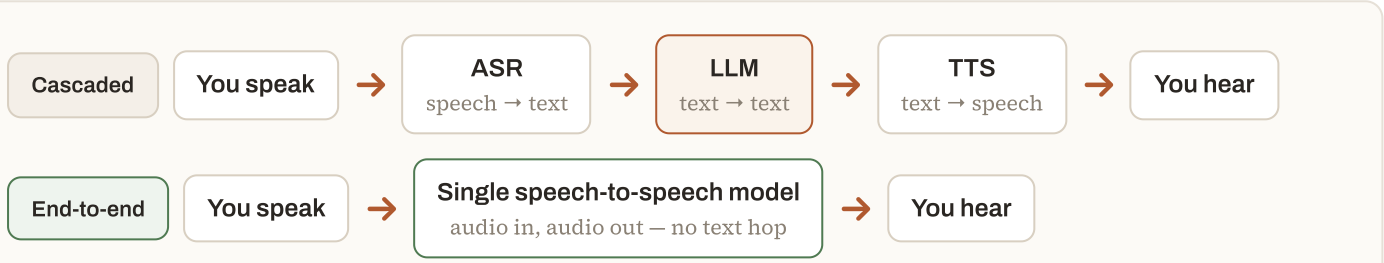
N Conversational & Multimodal AI

Let me explain why voice assistants feel laggy or snappy, and whether a model can really hear your tone.

N1 Latency considerations and architecture in conversational AI (Samvad, Sierra, GPT realtime, Google Agent Studio).

Madhav Bhetanabhotla

Voice AI feels natural only if the round-trip is short — humans notice pauses beyond roughly half a second. The challenge is that the classic architecture is a **pipeline of three models in series**, and every stage adds delay:



Two architectures. Cascaded (top) is modular but stacks three latencies and loses tone at the ASR step. End-to-end speech models (GPT realtime-style) skip the text hops for lower latency and richer tone.

SAMPLE MATRIX — WHERE THE LATENCY BUDGET GOES (CASCADED)

STAGE	ROUGH LATENCY	HOW IT'S REDUCED
Detect end-of-speech (VAD / endpointing)	100–300 ms	Tune silence thresholds; predict turn-end
ASR (speech → text)	~100–300 ms	**Stream** — transcribe while you're still talking
LLM first token (TTFT)	200–800 ms	Smaller/faster model, prompt caching, stream output
TTS (text → first audio)	100–300 ms	Streaming TTS — start speaking the first words early

KEY ARCHITECTURAL MOVES

Stream everything (don't wait for each stage to finish), handle **barge-in** (let the user interrupt), keep models close to the user (edge), cache the system prompt, and for the lowest latency + best tone, use an **end-to-end speech model** that skips the text round-trip entirely. Agentic products (e.g. Sierra) add tool calls and business logic on top, which is more latency to budget for.

IN ONE SENTENCE

"Conversational AI is usually ASR → LLM → TTS in series, so you fight latency by streaming every stage and handling interruptions — and newer end-to-end speech-to-speech models cut delay further by dropping the text hops."

N2 With audio input, how does the model understand tone — detect sarcasm or emotional context?

Ajay Sarswat

It hinges entirely on *which* architecture from N1 you use, because tone lives in the **sound**, not the words:

- **Cascaded (ASR → text → LLM):** the ASR step throws tone away. "Oh, great." becomes the three text tokens `Oh , great .` — the sarcastic sigh and flat pitch are gone before the LLM ever sees it. It can only guess tone from wording and context.
- **Audio-native (speech-to-speech):** the model ingests the raw audio, so it can pick up **prosody** — pitch, pace, loudness, pauses, sighs. That's what lets it distinguish an excited "great!" from a deadpan "great."

COMMON MISCONCEPTION

"If it hears me, it gets sarcasm." Prosody helps a lot, but sarcasm often needs *world knowledge and situation*, not just acoustics ("great, my flight's cancelled" is sarcastic regardless of pitch). Even people miss sarcasm over audio — so treat detected emotion as a useful signal, not a reliable fact, and design for graceful mistakes.

IN ONE SENTENCE

"Tone is carried by the audio's prosody, so a speech-native model can sense it while a transcribe-first pipeline loses it at the text step — and sarcasm stays hard because it often depends on context, not just how something sounds."

O Building, Deploying & Engineering Decisions

This is where I turn a demo into a product: where the LLM belongs, where rules belong, and what runs where.

O1 Can vibe/prompt coding produce production-ready code —

and how production-ready is the stock-analysis app we built?

Aman Anand · Anonymous

Prompt/"vibe" coding is fantastic for getting to a *working prototype* fast. But "works on my screen" and "production-ready" are different bars. AI-generated code is a strong first draft; production-readiness comes from the engineering you wrap around it.

SAMPLE MATRIX — PROTOTYPE VS PRODUCTION

CONCERN	PROTOTYPE (VIBE-CODED)	PRODUCTION NEEDS
Happy path works	Usually yes	Yes — plus every edge case
Error handling & retries	Often missing	Comprehensive
Tests	Rare	Automated, in CI
Security & secrets	Frequently unsafe	Reviewed, hardened
Maintainability	Variable	Readable, documented, owned

STRAIGHT ANSWER ON THE DEMO APP

It's a **prototype**, and should be treated as one. It demonstrates the pattern — LLM orchestration plus tool calls for the numbers (Section G3) — and is perfect for learning and demos. It is *not* production-ready: it lacks hardened error handling, auth, rate-limit and cost controls, input validation, financial-grade testing/backtesting, and compliance review. Ship the *idea*; re-engineer before real money or users touch it.

IN ONE SENTENCE

"Vibe coding gets you a working prototype fast, but production-ready means tests, error handling, security, and review on top — so the stock-analysis app is a great learning demo, not something to deploy as-is."

O2 When is it better to add another specialized LLM vs. continuing to scale/improve the existing one?

Sk Asif Akram

Default to **one capable general model** — fewer moving parts, less to maintain. Reach for a second, specialized model only when a concrete signal tells you the single model is the wrong tool for part of the job.

SAMPLE MATRIX — SIGNALS TO ADD ANOTHER MODEL

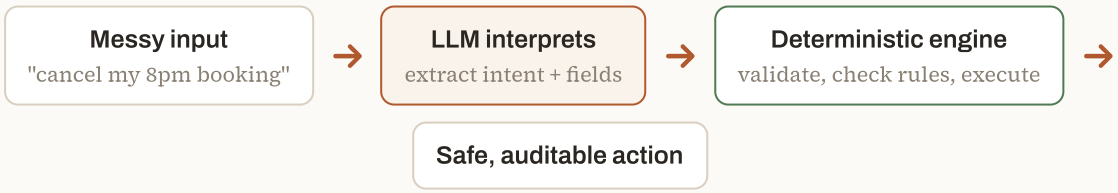
SIGNAL	BETTER MOVE
One sub-task is high-volume and simple	Route it to a small/cheap model (save cost + latency)
A narrow task needs a specific style/format reliably	Fine-tune a small model for just that
Different modality (images, speech, code)	Add a model specialized for it
Quality gap only on hard queries	Router: cheap model for easy, big model for hard
General model already meets the bar	Don't add anything — keep it simple

> **IN ONE SENTENCE**
>
> *"Stick with one general model until a clear pressure — cost, latency, a narrow skill, or a new modality — makes a smaller specialized model or a router the obviously cheaper, better fit for that slice of the work."*

Q3 To ship an AI feature you need deterministic (rule engine) and predictive (LLM) parts together — how do you decide the boundary, and what parameters matter?

Jishan Baig

The reliable division of labour: **the LLM handles the fuzzy, language, and judgement parts; deterministic code handles anything that must be exact, safe, and auditable.** A great default is *LLM proposes, code disposes* — the model interprets and suggests, then rule-based code validates and executes.



LLM proposes, code disposes. The fuzzy understanding is the model's job; the money/permissions/transaction is the rule engine's job.

SAMPLE MATRIX — WHICH SIDE OWNS IT, AND THE PARAMETERS THAT DECIDE

PARAMETER	PUSH TOWARD DETERMINISTIC WHEN…	PUSH TOWARD LLM WHEN…
Correctness	Must be exact/repeatable (money, permissions)	"Good enough" and fuzzy is fine
Auditability	You must explain every decision (compliance)	No strict audit trail needed
Input shape	Structured, known cases	Free text, messy, open-ended
Cost of a mistake	High / irreversible	Low / easily corrected
Token cost & latency	Hot path, tight budget	Occasional, tolerant

> **IN ONE SENTENCE**
>
> *"Let the LLM do the language and judgement and let deterministic code do anything that must be exact, safe, or auditable — draw the line by correctness, auditability, input messiness, cost of error, and token/latency budget."*

O4 What's the best way to automate a project-management workflow without AI, and do LLMs make it more efficient — or is it too early?

Ankit Sahu

Automate the *deterministic* parts first with plain tooling — that's mature and reliable: status transitions, assignment rules, reminders, templates, and integrations (Jira/Linear automation, GitHub Actions, Zapier/Make, cron scripts). This is the O3 principle applied to PM: rules for the repeatable mechanics.

Then add an LLM for the *fuzzy language layer* on top — and no, it isn't too early for these, as long as a human reviews the important calls:

WORKFLOW STEP	BEST HANDLED BY
Move ticket when PR merges; send reminders	Rules / automation
Summarise a long thread into an update	LLM
Draft a ticket from a messy Slack message	LLM (human confirms)
Triage & tag incoming issues	LLM suggests → rules enforce
Enforce required fields / SLAs	Rules

IN ONE SENTENCE

"Automate the repeatable mechanics with ordinary rule-based tooling, then layer an LLM onto the language-heavy bits (summaries, drafting, triage) with a human check — that combination is ready today, not too early."

O5 Which models can be run locally?

(raised in session)

The dividing line is **open-weight vs closed**. Open-weight models (weights published for download) can run locally; closed frontier models (GPT-4-class, Claude, Gemini) are **API-only** — you cannot run them on your own machine. What you *can* run locally depends on your memory (VRAM/RAM), and **quantization** (compressing weights to 4–8 bits) lets bigger models fit smaller hardware.

SAMPLE MATRIX — LOCAL FEASIBILITY

MODEL TYPE	EXAMPLES	RUNS LOCALLY?
Small open-weight (1–8B)	Llama, Mistral, Qwen, Gemma, Phi	Yes — laptop / single GPU, even CPU
Mid open-weight (13–70B)	Llama 70B, Mixtral	Yes with a strong GPU / quantization
Large open-weight (100B+)	DeepSeek, big MoE models	Only on serious multi-GPU rigs
Closed frontier	GPT-4-class, Claude, Gemini	**No** — API only

HOW

Tools like **Ollama**, **LM Studio**, and **llama.cpp** make local runs one command. Local = full privacy and no per-token bill, at the cost of managing hardware and (usually) lower ceiling than frontier models.

IN ONE SENTENCE

"Open-weight models (Llama, Mistral, Qwen, Gemma, Phi, DeepSeek) run locally — small ones on a laptop, big ones on serious GPUs — while closed frontier models like GPT-4-class, Claude, and Gemini are API-only."

P Model & Company Comparison

I'll show you how I compare models, what actually differs between labs, and whether I think the progress is real.

Amit Pandey · amit.s.pandey@gmail.com

P1 Which is better, "Fable" or "ChatGPT 5.6," and why would one be better for unknowns?

Amit Manchanda

(Using the two model names from the session as stand-ins.) There is no universal "better" — **"better" is always relative to *your* task.** The only honest way to choose is to run both against your own examples (an *eval set*) and measure. A model can top public leaderboards and still lose on your specific workload.

SAMPLE MATRIX — HOW TO ACTUALLY DECIDE

DIMENSION	HOW TO TEST IT FOR YOUR USE CASE
Quality on your task	Run 20–50 real examples through both; score them
Handling "unknowns"	Feed edge cases & unanswerable questions — does it say "I don't know" or bluff?
Cost & latency	Measure tokens & response time at your volume
Tools / context / modality	Does it support the tools, context length, inputs you need?

"Better for unknowns" usually means better *calibration*: when a model faces something outside its knowledge, the good behaviour is to express uncertainty, ask a clarifying question, or reach for a tool/search — rather than confidently hallucinate (Section G1). A model that admits "I'm not sure, let me check" is safer on unfamiliar territory than a more fluent one that bluffs.

> **IN ONE SENTENCE**
>
> *"There's no global winner — pick by testing both on your own examples, and 'better for unknowns' means better calibrated: it admits uncertainty or checks a source instead of confidently making things up."*

P2 Are all AI companies fundamentally building on the same core concepts, with the difference being training data and alignment/constraints?

Sk Asif Akram

Largely yes on the core, and the differences are a bit broader than just data + alignment. Nearly every lab uses the same foundations: the **Transformer**, **next-token prediction**, tokenization + embeddings, and the pretrain→fine-tune→align pipeline (Section A). The moat is in the execution:

SHARED BY ~EVERYONE	WHERE LABS DIFFER (THE MOAT)
Transformer architecture	Training **data** quality, mix & scale
Next-token prediction	**Alignment** / RLHF & safety constraints (the "personality")
Tokenize → embed → attention	Post-training for **reasoning**, tools, agents
Softmax + sampling	Scale, efficiency tricks, context length, multimodality, infra

> **IN ONE SENTENCE**
> *"Yes — everyone builds on the same Transformer + next-token core, and the real differences are data quality, scale, alignment, and post-training for reasoning and tools."*

P3 Is the progress partly an illusion, since tokenizing/embedding and O(n²) are unchanged since GPT-1?

Soham Kar

No — the primitives being stable doesn't make the capability gains fake. It's true that tokenization, embeddings, and quadratic attention are recognizably the same family as GPT-1. But enormous, *measurable* jumps came from everything built on top of those primitives.

> **ANALOGY**
> A 2026 car and a 1908 Model T both have four wheels, an engine, and a steering wheel. The primitives are "unchanged" — yet no one would call a century of automotive progress an illusion. Same primitives, transformed capability.

SAMPLE MATRIX — STABLE PRIMITIVES, DRAMATIC GAINS

UNCHANGED SINCE GPT-1	WHAT ACTUALLY DROVE THE LEAPS
Tokenize → embed	~1000× more parameters & data (scaling laws)
Transformer attention	RLHF & instruction tuning (usable assistants)
O(n²) attention (core)	Reasoning training, tool use, agents
Next-token objective	1M-token context, multimodality, efficiency tricks

> **IN ONE SENTENCE**
> *"The building blocks are stable, but scale, alignment, reasoning training, tools, context length, and multimodality turned them into something GPT-1 couldn't do — the progress is real, not an illusion."*

Q Demo Code & Course Logistics

Let me answer the questions about the sample scripts, plus the housekeeping ones. (My code answers describe the standard pattern — map them onto your actual file.)

Q1 Zero-shot code: will it give a different intent on each run or the same — and what if the intent it returns is wrong / doesn't match the ticket?

Anurag Thenge · Abinash Gupta

Whether it varies run-to-run is entirely a **sampling** question (Section K):

- **temperature = 0** (greedy) → effectively the *same* intent each run for the same input (tiny hardware nondeterminism aside).
- **temperature > 0** → it *can* vary between runs.

So for an intent-classification step, set a low temperature to keep it stable. **If the returned intent is wrong or doesn't match the ticket**, that's a reliability problem to engineer around — don't trust it blind:

FIX	WHAT IT DOES
Go few-shot	Add 2–5 examples of ticket → correct intent; accuracy jumps vs zero-shot
Constrain the output	Force intent to a fixed enum; reject anything off-list
Validate & retry	Check the intent against rules; re-ask if invalid
Confidence + human review	Ask it to flag low confidence; route those to a person

> **IN ONE SENTENCE**
>
> *"At temperature 0 it repeats the same intent; above 0 it can drift — and because it's sometimes wrong, pin it with few-shot examples, a fixed set of allowed intents, and validation rather than trusting a single zero-shot guess."*

Q2 Code walkthrough: is `ask(prompt)` where the LLM call happens, where is the Example variable passed, and where is "role" defined — or does the LLM just understand it?

Anonymous · Padmanabha Kulkarni · Anese Syed

In the standard structure of a script like this:

- **Yes — `ask(prompt)` is the wrapper where the real API call happens.** Inside it you'll find the SDK call (e.g. `client.messages.create(...)` / `chat.completions.create(...)`). That line is the actual LLM invocation; everything else just builds the input or handles the output.
- **The Example variable is passed by being *concatenated into the prompt text*** before `ask()` is called. Few-shot examples aren't a special API field — they're just text you paste into the message the model reads.
- **"role" is a structural field in the messages array** you send — `{"role": "system", ...}`, `{"role": "user", ...}`. You define it; the model was *trained* to treat those roles differently, so it "understands" them because the format was baked in during training, not because it infers them at runtime.

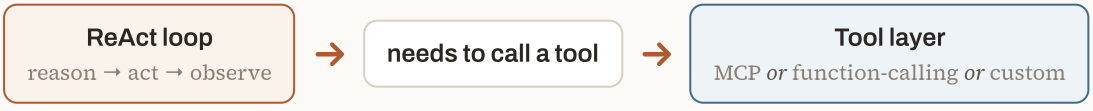
```
messages = [
  {"role": "system",  "content": SYSTEM_PROMPT},   # role defined here
  {"role": "user",    "content": EXAMPLES + "\n" + ticket}  # Example folded into the text
]
def ask(messages):
    return client.messages.create(model=..., messages=messages)  # ← the LLM call
```

> **IN ONE SENTENCE**
>
> *" `ask()` is where the API call fires, few-shot examples ride in as plain text concatenated into the prompt, and 'role' is a field you set on each message that the model was trained to respect — it doesn't infer it on its own."*

Q3 Does the ReAct prompt use MCP tools internally?

Ramasamy A

Not necessarily — they're independent layers that are easy to conflate. **ReAct** (Reason + Act) is a *prompting pattern*: the model alternates a thought ("I should look up X") with an action (call a tool), reads the result, and repeats. **MCP** (Model Context Protocol) is a *standard for connecting* tools and data sources to a model. A ReAct loop can get its tools from MCP — or from plain function-calling, or from custom string parsing. MCP is one plumbing option; ReAct is the reasoning loop on top.



Orthogonal layers. ReAct decides *when* to use a tool; MCP is one way to *expose* that tool. Using ReAct doesn't imply MCP.

IN ONE SENTENCE

"No — ReAct is the reason-then-act loop and MCP is one way to connect the tools it calls; a ReAct agent may use MCP, function-calling, or custom glue, so the two aren't the same thing."

Q4 The venv commands aren't showing in the chat window.

Sudhanshu Saxena

Virtual-environment setup is a **terminal** activity, not a chat one — the commands run in your shell, and their effect (an activated env) won't appear as chat messages. Run them in an actual terminal:

```
python -m venv venv                 # create
source venv/bin/activate            # activate (macOS/Linux)
venv\Scripts\activate               # activate (Windows)
pip install -r requirements.txt     # install deps
```

IF THEY STILL DON'T SHOW

You're likely typing them where output isn't echoed, or the tool routed them to a hidden shell. Open the integrated terminal directly and run them there; confirm activation by the `(venv)` prefix on your prompt. (This is a setup/logistics hiccup — flag it to the course team with your OS and tool if it persists.)

Q5 Do we get API keys / Claude or OpenAI subscriptions as part of this course?

Anonymous

This is a **course-administration question**, not a technical one, so confirm it with the organizers — I can't speak for the program's policy. Typically it's one of these arrangements:

ARRANGEMENT	WHAT IT MEANS FOR YOU
Course provides keys/credits	Temporary key or shared credits for the duration
Bring your own key (BYOK)	You create a provider account and add a little billing (API use is pay-as-you-go, usually cents for the exercises)
Local / open model	No key needed — run a local model (Section O5)

IN ONE SENTENCE

"That's up to the course — please confirm with the organizers; the usual options are provided keys/credits, bring-your-own-key with small pay-as-you-go billing, or using a free local model."

R Course Pace, Materials & Wellbeing

I know the session moved fast and assumed some ML background — so here's the path I'd take to backfill it, plus how I'd revise and refocus.

R1 The session hit information-overload and assumed a neural-network background (FFN in ~1 minute). For those from a software background, what books / materials / learning path build the foundation?

Tarun Kumar · Narsi Veldanda · Anonymous

Completely fair — a live session has to compress things like feed-forward networks into a sentence. The good news: as a software engineer you're well-placed to learn this *by building*, and there's a well-worn path. Go in layers — intuition first, mechanics later — and don't try to hold it all at once.

SAMPLE MATRIX — A STAGED LEARNING PATH

STAGE	RESOURCE	FORMAT	ROUGH TIME
1. Intuition	3Blue1Brown — "Neural Networks" series & "But what is a GPT?"	Video	3–4 hrs
2. See the Transformer	Jay Alammar — "The Illustrated Transformer"	Blog	1–2 hrs
3. Build it yourself	Andrej Karpathy — "Neural Networks: Zero to Hero" (micrograd → nanoGPT)	Video + code	20–30 hrs
4. LLM end-to-end	Sebastian Raschka — "Build a Large Language Model (From Scratch)"	Book	Weeks
5. Broad ML base	Aurélien Géron — "Hands-On Machine Learning"	Book	Weeks
6. Applied / prompting	DeepLearning.AI short courses; Hugging Face course	Courses	Ongoing

KEY TAKEAWAY

If you do only one thing, do **Karpathy's "Zero to Hero."** It's built for engineers, starts from scratch, and by the end you'll have coded a tiny GPT — after which every term in this guide (FFN, attention, embeddings) is something you've *implemented*, not just heard.

IN ONE SENTENCE

"Backfill it by building: 3Blue1Brown for intuition, the Illustrated Transformer to see it, then Karpathy's 'Zero to Hero' to code a small GPT yourself — layer intuition before mechanics instead of absorbing everything at once."

R2 What is the exam scope and practice?

Amit Manchanda

The exact scope, format, and weighting come from the **course organizers** — please confirm with them, as I can't set or speak for it. What I *can* offer: this guide maps closely to the concepts a fundamentals exam on this material would likely test, so it doubles as a revision sheet.

HOW TO REVISE WITH THIS GUIDE

Read each section's *"In one sentence"* box first — those are exam-ready summaries. Then cover the answer and try to reproduce the diagram or the sample matrix from memory (e.g. training vs inference, top-k vs top-p, RAG vs fine-tuning). The "Common misconception" boxes are classic trick-question territory — know why each wrong belief is wrong.

IN ONE SENTENCE

"Confirm the actual scope with the organizers — but this guide's one-sentence summaries, matrices, and misconception boxes are a ready-made revision set for the fundamentals it's likely to cover."

R3 How to regain deep focus?

Shubh Mandalia

First, the reassuring part: feeling scattered after a dense session is *normal* — it's cognitive overload, not a personal failing. New, abstract material is genuinely tiring, and focus is a skill you rebuild, not a switch you flip.

PRACTICE	WHY IT WORKS
Single-task in focused blocks (e.g. 25 min, then break)	Removes the switching cost that shreds attention
Kill inputs — phone away, notifications off, one tab	Each interruption costs minutes to recover from
Learn in layers, spaced over days	Beats cramming; memory consolidates with rest & repetition
Protect sleep, movement, daylight	Focus is downstream of basic physiology
Learn by doing (code a tiny example)	Active engagement holds attention far better than passive reading

IN ONE SENTENCE

"Overload after a dense session is expected — rebuild focus with short single-tasked blocks, fewer interruptions, spaced learning over cramming, and enough sleep, and let hands-on practice carry your attention."

S Bonus Deep-Dive: Inside a BERT Encoder

Let me trace one short sentence all the way through a real encoder layer — with numbers at every step.

This section is a single end-to-end worked example. We'll push the sentence **"the cat sat"** through one **BERT-base encoder layer** and watch the numbers change at each stage. BERT-base uses `d_model = 768` dimensions, **12 attention heads** (64 dims each), a **3072**-wide feed-forward network, and **12 stacked encoder layers**. To keep the matrices readable we show only the first few of the 768 columns and write `…` for the rest — every "…" hides 760-odd real numbers.

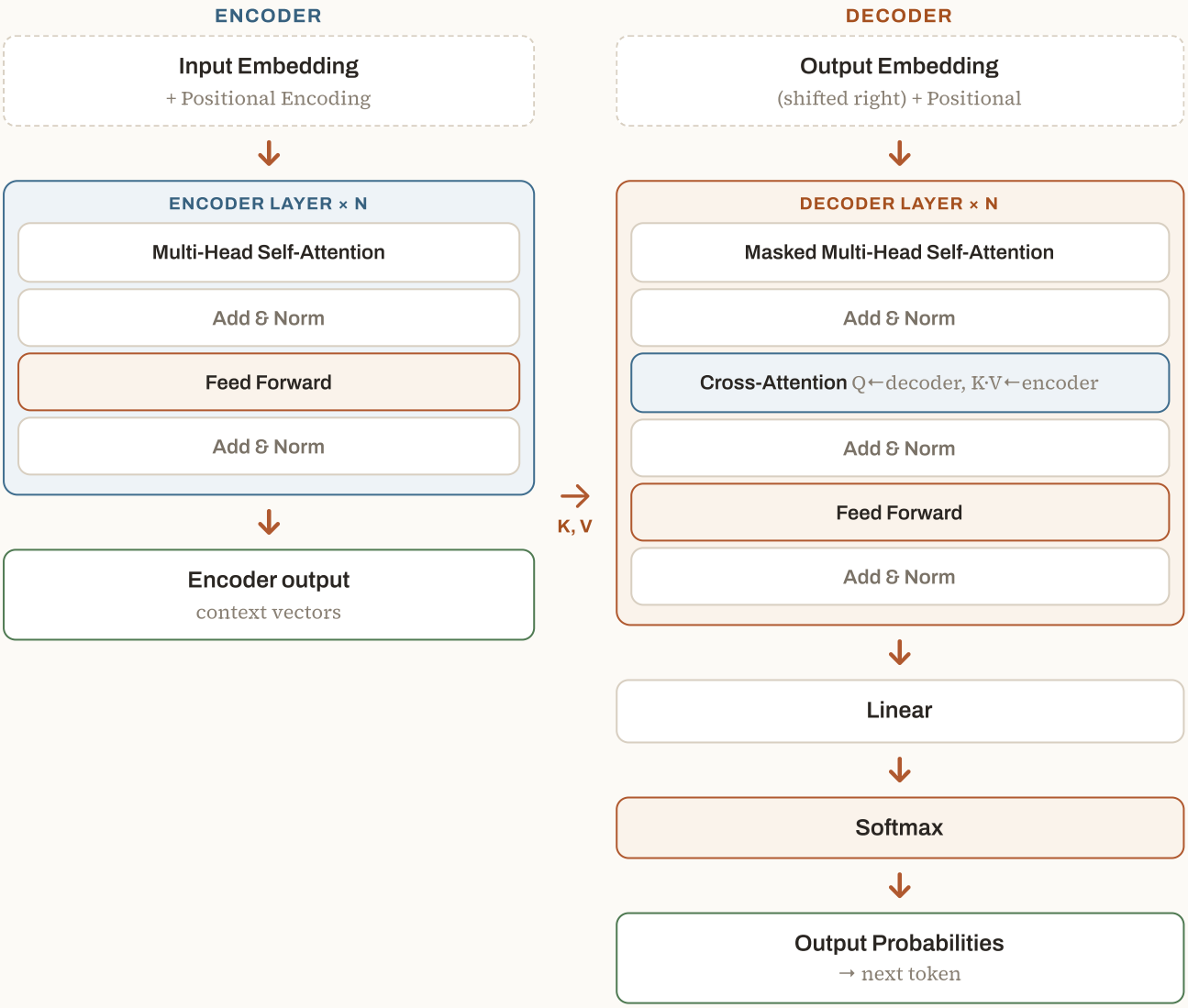
ENCODER VS DECODER — ONE KEY DIFFERENCE

An **encoder** (BERT) reads the *whole* sentence at once — every token attends to every other token, left *and* right (bidirectional). A **decoder** (GPT) only lets a token see tokens *before* it, because it's generating left-to-right. Same block, but the encoder has no such mask. That's why BERT is used for *understanding* (classification, search) and GPT for *generation*.

Amit Pandey · amit.s.pandey@gmail.com

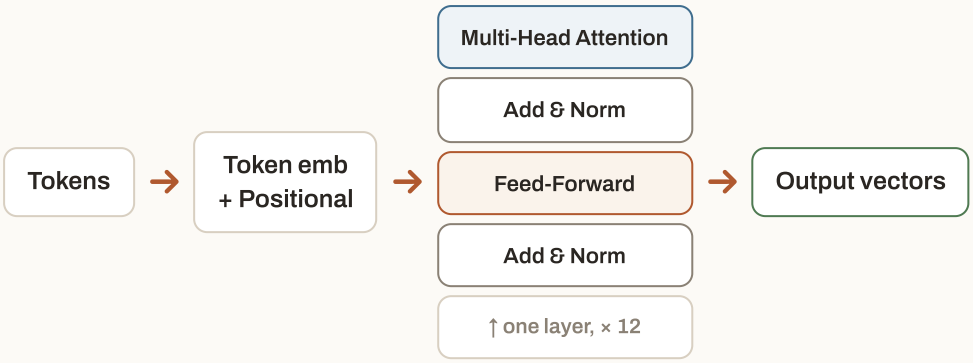
The classical Transformer — the full picture

Before we zoom in, here is the entire 2017 architecture. It has **two towers**: an **encoder** (left) that reads and understands the input, and a **decoder** (right) that writes the output one token at a time. Each shaded block repeats `N×` (N = 6 in the original paper, 12 in BERT-base), and — the idea to hold onto — *each layer's output is the next layer's input*.



The original 2017 Transformer. The encoder's final output is fed into *every* decoder layer's cross-attention as its Keys and Values — the "K, V" arrow is the bridge between the towers. **BERT** keeps only the left tower; **GPT** keeps only the right. We trace the encoder in full detail next, then follow the decoder tower to the final output.

Amit Pandey · amit.s.pandey@gmail.com



The encoder layer we're about to trace. Input becomes vectors, then flows through attention → add&norm → feed-forward → add&norm. BERT-base repeats this block 12 times; we'll walk one.

S1 Step 1 — Tokenize the sentence into ids

WordPiece splits the text and looks each piece up in BERT's 30,522-word vocabulary. BERT also wraps the sentence in two special tokens: `[CLS]` at the front (its final vector represents the whole sentence) and `[SEP]` at the end. To keep the maths clean we'll trace just the three content tokens.

TOKEN	[CLS]	THE	CAT	SAT	[SEP]
VOCABULARY ID	101	1996	4937	2938	102

S2 Step 2 — Look up token embeddings

Each id indexes one row of the embedding matrix `[30522 × 768]` (Section I). That gives every token a 768-number vector. (BERT also adds a small *segment* embedding for sentence-pair tasks; we omit it here.)

TOKEN EMBEDDINGS · SHAPE 3 × 768

	DIM 0	DIM 1	DIM 2	DIM 3	…	DIM 767
THE	0.021	-0.014	0.033	0.009	…	0.007
CAT	-0.045	0.052	0.011	-0.030	…	-0.023
SAT	0.008	0.019	-0.041	0.022	…	0.030

S3 Step 3 — Build the positional encoding matrix

Attention has no built-in sense of order, so we add position information of the *same* shape `[3 × 768]`. The original Transformer computes it with sines and cosines (BERT actually *learns* these vectors, but the shape and role are identical):

$$PE(pos,\ 2i) = \sin\left(\ pos\ /\ 10000^{2i/768}\ \right)$$
$$PE(pos,\ 2i+1) = \cos\left(\ pos\ /\ 10000^{2i/768}\ \right)$$

POSITIONAL ENCODING · SHAPE 3 × 768 (VALUES ROUNDED)

POSITION	DIM 0	DIM 1	DIM 2	DIM 3	…	DIM 767
0 (THE)	0.000	1.000	0.000	1.000	…	1.000
1 (CAT)	0.841	0.540	0.829	0.560	…	1.000
2 (SAT)	0.909	-0.416	0.921	-0.389	…	1.000

Notice position 0's first dims are exactly `sin(0)=0` and `cos(0)=1` — an easy sanity check.

S4 Step 4 — Add them: the encoder's input matrix X

Element-wise add the two matrices (token embedding + positional). The result **X** is what actually enters the first encoder layer — each row is a token that now carries both *meaning* and *position*.

X = TOKEN EMB + POSITIONAL · SHAPE 3 × 768

	DIM 0	DIM 1	DIM 2	DIM 3	…	DIM 767
THE	0.021	0.986	0.033	1.009	…	1.007
CAT	0.796	0.592	0.840	0.530	…	0.977
SAT	0.917	-0.397	0.880	-0.367	…	1.030

CHECK THE ARITHMETIC

"cat" is at position 1. Its dim 0 = token `-0.045` + positional `0.841` = **0.796**. Its dim 1 = `0.052` + `0.540` = **0.592**. Every cell of X is just that sum.

S5 Step 5 — Project X into Q, K, V

Inside the attention block, X is multiplied by three learned weight matrices to produce **Queries**, **Keys** and **Values** (Section J). Each weight matrix is `[768 × 768]`:

$$Q = X \cdot W_Q \qquad K = X \cdot W_K \qquad V = X \cdot W_V$$

BERT-base then **splits** each 768-wide vector into **12 heads of 64 dims**, and attention runs independently in each head. Below is the maths for *one* head (so $d_k = 64$); the other 11 run in parallel.

Amit Pandey · amit.s.pandey@gmail.com

Q, K, V FOR HEAD 1 · SHAPE 3 × 64 EACH

	Q (QUERY)			K (KEY)			V (VALUE)		
	0	1	…	0	1	…	0	1	…
THE	0.12	-0.08	…	0.10	-0.06	…	0.50	0.10	…
CAT	0.30	0.11	…	0.28	0.09	…	-0.10	0.40	…
SAT	-0.04	0.22	…	-0.02	0.19	…	0.20	-0.20	…

S6 Step 6 — Attention: scores → scale → softmax → context

Now the core formula runs. First every Query dots with every Key (using all 64 dims of the head) to make a 3×3 score grid, then we divide by $\sqrt{d_k} = \sqrt{64} = 8$ to keep it stable:

$$\text{Attention} = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$

$Q \cdot K^T$ RAW SCORES (3×3)

	THE	CAT	SAT
THE	6.4	4.0	3.2
CAT	4.1	7.2	5.0
SAT	3.0	5.6	6.8

÷ 8, THEN SOFTMAX EACH ROW (WEIGHTS)

	THE	CAT	SAT
THE	**0.42**	0.31	0.28
CAT	0.28	**0.41**	0.31
SAT	0.25	0.35	**0.40**

Each row of the right-hand matrix sums to 1 — these are attention weights. Read row "sat": it attends 0.40 to itself, 0.35 to "cat", 0.25 to "the" — so "sat" pulls most from "cat" (its subject).

Finally, multiply the weights by V to blend the Values. Row "the" = $0.42 \cdot V_{\text{the}} + 0.31 \cdot V_{\text{cat}} + 0.28 \cdot V_{\text{sat}}$, and so on:

HEAD-1 CONTEXT = WEIGHTS · V · SHAPE 3 × 64

	DIM 0	DIM 1	DIM 2	DIM 3	…	DIM 63
THE	0.232	0.109	0.117	0.060	…	0.041
CAT	0.160	0.129	0.183	-0.009	…	0.052
SAT	0.171	0.083	0.220	0.012	…	0.037

CHECK THE BLEND

Context "the", dim 0 = 0.42×0.50 + 0.31×(−0.10) + 0.28×0.20 = 0.210 − 0.031 + 0.056 = **0.232.** That single number is a weighted mix of all three tokens' Values — which is exactly how "the" becomes context-aware.

MULTI-HEAD, THEN COMBINE

All **12 heads** produce their own 3×64 context. They're **concatenated** back into 3×768 and passed through one more learned matrix W_O `[768 × 768]`. The result is the attention block's output — call it **Attn(X)**, shape `3 × 768`.

S7 Step 7 — Add & Norm (residual + LayerNorm)

Two things happen. First a **residual connection** adds the block's input back to its output — this lets gradients flow and stops information being lost: `Z = X + Attn(X)`. Then **LayerNorm** rescales each token's row to a stable range:

$$\text{LayerNorm}(z) = \gamma \cdot (z - \mu) / \sqrt{(\sigma^2 + \varepsilon)} + \beta$$

where `μ` and `σ` are the mean and standard deviation *of that one row* (across all 768 dims), and `γ`, `β` are learned scale/shift vectors. So each token vector is re-centred to mean 0, variance 1, then reshaped.

Z = X + ATTN(X) · SHAPE 3 × 768

	DIM 0	DIM 1	DIM 2	DIM 3	…
THE	0.253	1.095	0.150	1.069	…
CAT	0.956	0.721	1.023	0.521	…
SAT	1.088	-0.314	1.100	-0.355	…

H_1 = LAYERNORM(Z) · EACH ROW NOW MEAN ≈ 0, VAR ≈ 1

	DIM 0	DIM 1	DIM 2	DIM 3	…
THE	-0.71	1.02	-0.83	0.98	…
CAT	0.34	-0.12	0.51	-0.44	…
SAT	0.88	-1.03	0.90	-1.10	…

Amit Pandey · amit.s.pandey@gmail.com

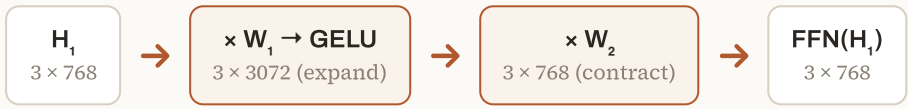
WHY NORMALIZE

After adding two matrices, values can drift to different scales across tokens. LayerNorm puts every token's 768 numbers on the same footing (mean 0, var 1) so the next layer sees consistent inputs — this is what makes deep 12-layer stacks trainable at all.

S8 Step 8 — Feed-Forward Network (position-wise)

Now each token is processed *on its own* (no mixing between tokens here) by the same two-layer network. It expands 768 → 3072, applies the **GELU** activation, then contracts 3072 → 768:

$$\mathrm{FFN}(h) = \mathrm{GELU}(\, h \cdot W_1 + b_1 \,) \cdot W_2 + b_2$$



The feed-forward "thinking" step. W_1 is `[768 × 3072]`, W_2 is `[3072 × 768]`. The wide middle layer is where much of the model's learned knowledge sits.

FFN(H₁) · SHAPE 3 × 768

	DIM 0	DIM 1	DIM 2	DIM 3	…
THE	0.14	-0.22	0.31	0.08	…
CAT	-0.09	0.27	0.12	-0.18	…
SAT	0.21	-0.05	0.34	0.02	…

S9 Step 9 — Second Add & Norm → the layer's output

The same residual + LayerNorm as Step 7, now around the feed-forward block: `H₂ = LayerNorm( H₁ + FFN(H₁) )`. That **H2** is the finished output of this encoder layer — three context-rich 768-vectors, one per token:

H₂ = ENCODER-LAYER OUTPUT · SHAPE 3 × 768

	DIM 0	DIM 1	DIM 2	DIM 3	…	DIM 767
THE	-0.58	0.79	-0.44	1.02	…	0.13
CAT	0.41	0.09	0.63	-0.52	…	-0.07
SAT	0.94	-0.88	1.05	-0.97	…	0.22

Amit Pandey · amit.s.pandey@gmail.com

WHAT HAPPENS NEXT — AND WHAT THE OUTPUT IS FOR

H_2 becomes the *input* to the next encoder layer, and this whole block repeats **12 times** in BERT-base. After the final layer: the `[CLS]` token's 768-vector is used as the **whole-sentence embedding** (feed it to a classifier for sentiment, spam, intent…), and each word's vector is its **contextual embedding** (used for tasks like named-entity recognition or as features for search).

S10 Step 10 — How the layers feed into one another

We traced *one* encoder layer. BERT-base has 12, and they simply stack: **each layer's output matrix becomes the next layer's input matrix.** The shape never changes — it stays `3 × 768` the whole way up — which is exactly why layers stack cleanly with no glue. This unchanging river of vectors is often called the **residual stream.**

Same shape in, same shape out — 12 times. Early layers capture surface patterns (grammar, nearby words); deeper layers build abstract meaning. Each layer refines what the one below handed it — it never restarts from scratch.

ANALOGY

An assembly line: every station receives the product from the one before, adds a little, and passes it on — same conveyor, same box size, progressively more finished. That hand-off from station to station is exactly how one layer feeds the next.

S11 Step 11 — The decoder half: generating an output

BERT stops at the encoder. The *classical* Transformer keeps going into the decoder to actually **produce** text — say, translating "the cat sat" into French. The decoder reuses every mechanic you've just seen (embeddings, attention, Add&Norm, feed-forward) and adds **two ideas:**

- **Masked self-attention.** As it writes, each position may attend only to tokens *already produced*, never the future — you can't look at a word you haven't written yet. The 3×3 attention grid from Step 6 becomes a lower *triangle* (future cells are set to $-\infty$ before softmax, so they get weight 0).
- **Cross-attention — the bridge.** A second attention step where the **Queries come from the decoder** but the **Keys and Values come from the encoder's output** (Step 9's context vectors). This is how the French words "look back at" the English sentence — it's the `K, V` arrow between the two towers.

So one decoder layer is: masked self-attention → Add&Norm → cross-attention → Add&Norm → feed-forward → Add&Norm. Stack it N times, exactly like the encoder.

S12 Step 12 — Linear + Softmax → the output, token by token

After the final decoder layer, each position has a 768-vector. One last **Linear** layer projects it up to the size of the whole vocabulary, and **Softmax** (Section J2) turns those scores into probabilities — the model's guess for the next word:

$$\text{logits} = h \cdot W_{\text{vocab}} \quad (768 \to \sim 30{,}000) \quad \to \quad \text{softmax}(\text{logits})$$

NEXT-TOKEN PROBABILITIES (FIRST FRENCH WORD)

CANDIDATE	LE	LA	UN	CHAT	…
PROBABILITY	**0.62**	0.15	0.08	0.04	…

Pick the top token (`le`), append it to the decoder's input, and run the whole decoder again to get the *next* word — this is the **autoregressive loop**. It repeats until the model emits the end-of-sequence token `[EOS]` .

Start to finish. The source is encoded once; the decoder then generates the translation one token at a time, each step feeding the next, until `[EOS]` .

IN ONE SENTENCE

"Start to finish: the encoder turns the input into context vectors through N stacked layers that each feed the next; then the decoder generates the output one token at a time — masked self-attention so it can't peek ahead, cross-attention back to the encoder's output, and a final Linear + Softmax that gives the next-token probabilities — looping until it emits [EOS]."

T Bonus Deep-Dive: From Enter Key to Answer

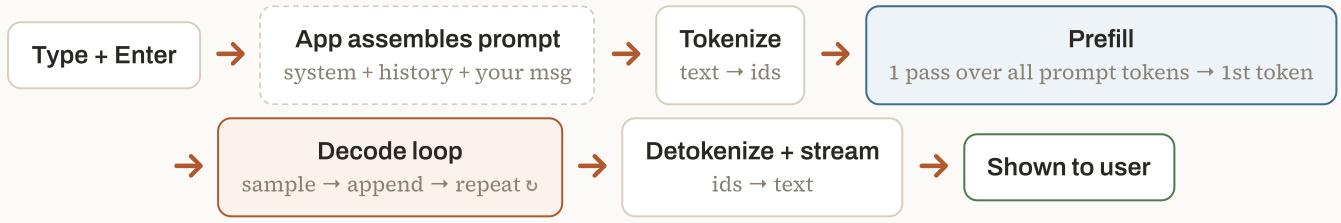
I'll walk you through everything that happens between pressing Enter and seeing the reply, on a real chat model.

Amit Pandey · amit.s.pandey@gmail.com

Section S showed the maths *inside* one layer. This section zooms out to the **whole journey of a single request** through a decoder-only chat model (GPT-style), and actually generates a few words so you can watch tokens appear one at a time. Our example:

THE REQUEST WE'LL TRACE

System instruction: "You are a concise geography assistant. Answer in one short sentence."
User types: "What's the capital of France?" then presses **Enter**.
What comes back: "The capital is Paris." — generated one token at a time.



The full round-trip. The heavy lifting is two phases inside the model — *prefill* (read the whole prompt at once) and *decode* (write the answer one token at a time). Everything else is packing text in and unpacking it out.

T1 Step 1 — The instant you hit Enter: the app builds the prompt

The model remembers nothing between calls (Section D), so before anything reaches it, the *app* assembles one complete prompt: the **system instruction** + the **prior conversation** (none yet here) + your **new message**. Chat models wrap each part with special role markers so the model can tell who said what:

ASSEMBLED MESSAGES (WHAT THE APP SENDS)

ROLE	CONTENT (WRAPPED AS `<|ROLE|>…<|END|>`)
SYSTEM	You are a concise geography assistant. Answer in one short sentence.
USER	What's the capital of France?
ASSISTANT	*(empty — the model fills this in)*

COMMON MISCONCEPTION

"The system instruction is set once, server-side." No — it's re-sent as tokens at the *front of every request* (and re-billed every time, Section M). It shapes the answer purely by being tokens the reply attends to (Section J1) — which is why a longer system prompt costs more on every single turn.

Amit Pandey · amit.s.pandey@gmail.com

T2 Step 2 — Tokenize the whole prompt

The entire assembled text (role markers included) is turned into token ids by the model's tokenizer (Section I). These are the **input tokens** you're billed for.

TOKENIZING THE PROMPT · (IDS ILLUSTRATIVE)

PIECE	TOKENS	IDS
system instruction	`You· are· a· concise· geography· assistant· .· Answer· in· one· short· sentence· .`	13 tokens
user message	`What· '· s· the· capital· of· France· ?`	8 tokens
role markers	`◁system▷` `◁user▷` `◁assistant▷`	~5 tokens
TOTAL INPUT	**≈ 26 tokens** enter the model	

T3 Step 3 — Prefill: read the whole prompt, predict the first token

All ~26 tokens flow through the decoder stack together (embeddings + positional + N layers of masked self-attention — the machinery from Section S). This is the **prefill** phase. Two things come out of it:

- The **KV cache** — the Keys and Values for every token at every layer are computed once and *stored*, so later steps don't recompute them.
- The model takes the *last* position's final 768-vector, runs it through Linear + Softmax (Section S12), and gets probabilities for the **first answer token**:

FIRST-TOKEN PROBABILITIES (AFTER THE PROMPT)

CANDIDATE	THE	PARIS	IT	FRANCE	…
PROBABILITY	**0.48**	0.31	0.09	0.05	…

Notice the system instruction already did its job: "answer in one short sentence" pushes probability toward a full-sentence opener like "The" rather than the bare word "Paris."

SAMPLE MATRIX — THE TWO PHASES OF GENERATION

	PREFILL	DECODE
PROCESSES	All prompt tokens at once	One new token at a time
SPEED	Parallel — fast per token	Sequential — one pass per token
KV CACHE	Builds it	Reuses + extends it
YOU FEEL IT AS	Time-to-first-token	The words streaming out

T4 Step 4 — Sample the first token

The probabilities are passed to the sampler (Section K). At a low temperature — right for a factual answer — it picks the top candidate:

Probabilities → one token. Greedy/low-temp picks "The"; a higher temperature might occasionally start with "Paris" or "It" instead (Section K).

"The" is now the first token of the answer. But one word isn't a sentence — so it gets fed back in, and the loop begins.

T5 Step 5 — The decode loop: one token at a time

Now the model repeats a tight loop: **append the token just produced, do a forward pass for only that new token** (reusing the KV cache so it doesn't re-read the whole prompt), get the next token's probabilities, sample, repeat. Watch the answer build:

THE AUTOREGRESSIVE LOOP · EACH ROW = ONE FORWARD PASS

STEP	SEQUENCE SO FAR	TOP NEXT TOKEN	PROB
1 (prefill)	[prompt]	**The**	0.48
2	[prompt] The	**capital**	0.55
3	[prompt] The capital	**is**	0.62
4	[prompt] The capital is	**Paris**	0.89
5	[prompt] The capital is Paris	**.**	0.71
6	[prompt] The capital is Paris .	**[EOS]**	0.80

Amit Pandey · amit.s.pandey@gmail.com

WHY THE KV CACHE MATTERS

Without it, step 5 would re-process all ~30 tokens from scratch; with it, only the single new token "Paris" is computed, and it attends to the stored Keys/Values of everything before. That's what makes streaming feel steady instead of slowing down with every word — and it's why the "decode" phase is memory-bound (Section H's KV cache, N1's latency).

T6 Step 6 — Detokenize, stream, and stop → what the user sees

Each generated id is immediately **detokenized** (id → text piece) and **streamed** to the screen — which is exactly why you see the reply appear word-by-word rather than all at once. Generation halts on a **stop condition**:

- the model emits the end-of-sequence token `[EOS]` (as in step 6 above), or
- it hits `max_tokens`, or a custom stop sequence.

The five generated tokens (`The capital is Paris .`) are the **output tokens** you're billed for (Section M), and they're joined back into text:

The finished reply, shown to the user. This assistant turn is also appended to the conversation, so it rides along as history in the *next* request (Section D).

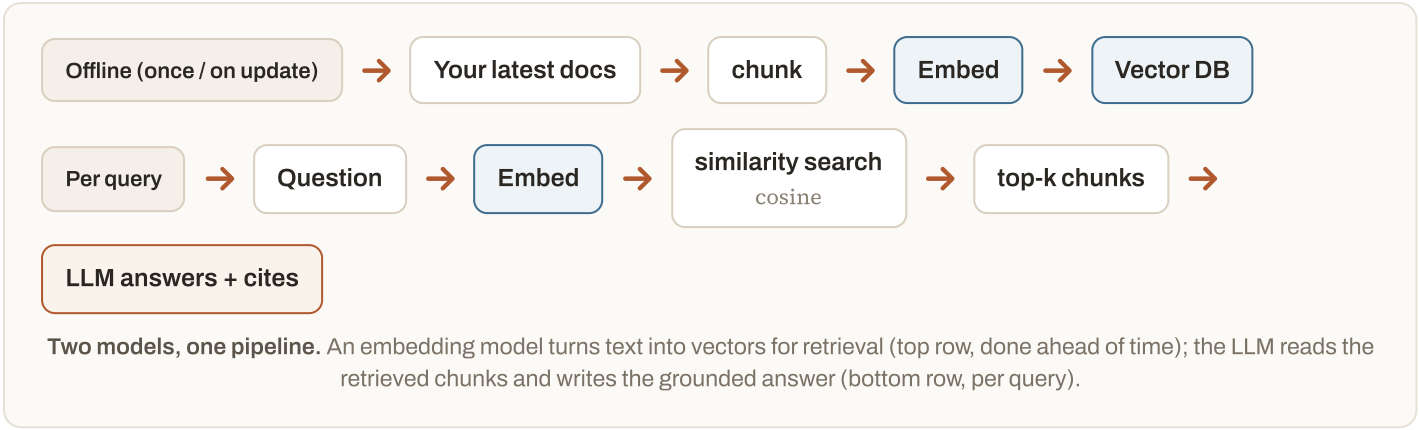
IN ONE SENTENCE

"When you hit Enter, the app bundles the system instruction + history + your message into one tokenized prompt; the model prefills it in one pass to predict the first token, then decodes the rest one token at a time (reusing a KV cache), and each token is detokenized and streamed to your screen until it emits [EOS] — that stream is the answer you read."

U Bonus Deep-Dive: RAG — Scoring a Query & Grounding on Fresh Facts

Here I'll show you how retrieval actually scores your query, and exactly where I inject the latest facts so the model can use them.

The whole point of RAG here is **grounding**: letting the model answer from your *latest / private* facts (which aren't in its weights — Section C) by putting the right passages into its prompt. Here's the full stack:



Two models, one pipeline. An embedding model turns text into vectors for retrieval (top row, done ahead of time); the LLM reads the retrieved chunks and writes the grounded answer (bottom row, per query).

U1 Step 1 — How retrieval scores your query

Scoring is a **similarity measurement in vector space.** The query is embedded into the *same* space as your chunks, then compared to each chunk with **cosine similarity** — the cosine of the angle between the two vectors. Same direction → score near 1 (very relevant); unrelated → near 0.

```
similarity(A, B) = (A · B) / (|A| · |B|)   → just A · B for unit-length (normalized) vectors
```

WORKED EXAMPLE — SCORING A REAL QUERY

Knowledge base has three chunks; the user asks **"What was our Q3 revenue?"**. We embed the query and each chunk (toy 4-dim vectors shown; real embedding vectors are typically 384–1024-dim and often unit-length, so the dot product below *is* the cosine similarity):

`QUERY VECTOR Q = [0.12, 0.88, 0.20, 0.40]`

CHUNK	VECTOR (TOY 4-DIM)	Q · CHUNK	SIMILARITY	RANK
C1 "Q3 FY26 revenue was $23.8M…"	[0.15, 0.85, 0.25, 0.42]	.018+.748+.05+.168	**0.98**	1
C3 "Q2 FY26 revenue was $19.1M."	[0.20, 0.80, 0.22, 0.45]	.024+.704+.044+.18	0.95	2
C2 "The Bengaluru office moved in June."	[0.60, 0.10, 0.70, 0.30]	.072+.088+.14+.12	0.42	3

With **top-k = 2**, chunks C1 and C3 are retrieved; C2 (about the office move) scores low and is dropped. Note the query never contained the word "$23.8M" — it matched on *meaning* ("revenue"), which is exactly what embeddings buy you over keyword search (Section I3).

RERANKING — THE OPTIONAL SECOND PASS

Cosine retrieval is fast but coarse. Production stacks often add a **reranker** (most embedding providers offer one): it takes the query + the top ~20 candidates and re-scores them more carefully, pushing the truly best chunk to the top. Cheap first pass to narrow, precise second pass to order.

U2 Step 2 — Where the grounding happens

Grounding happens at **one specific place: the prompt you send to the model**. The retrieved chunks are pasted into the prompt as `CONTEXT`, *before* the model generates anything, with an instruction to answer only from that context and cite it. The model then does ordinary generation (Section T) — but now the freshest facts are sitting right in front of it.

THE GROUNDED PROMPT THE MODEL ACTUALLY RECEIVES

```
System: Answer the QUESTION using ONLY the CONTEXT below. Cite the
source of each fact. If the CONTEXT lacks the answer, say so — do
not use outside knowledge.

CONTEXT:
  [C1] (finance-q3.pdf)  Q3 FY26 revenue was $23.8M, up 12% QoQ.
  [C3] (finance-q2.pdf)  Q2 FY26 revenue was $19.1M.

QUESTION: What was our Q3 revenue?
```

Two things make this "grounding," not guessing: the facts arrive through the *context* (so they can be as fresh as your last document upload — Section C), and the instruction forbids the model from answering off its own weights. In code it's just two calls:

```
qvec   = embed(query)                              # embedding model
hits   = vectordb.search(qvec, top_k=2)            # cosine similarity (U1)
prompt = GROUNDED_TEMPLATE.format(context=hits, question=query)
answer = llm.generate(prompt)                      # the LLM generates (T)
```

THE MODEL'S GROUNDED, CITED ANSWER

> "Q3 FY26 revenue was **$23.8M**, up 12% quarter-over-quarter." [source: finance-q3.pdf]

WHY GROUND INSTEAD OF FINE-TUNE FOR FRESH FACTS

For *facts that change* (revenue, prices, policies, news), grounding via RAG beats fine-tuning every time (Section L): you update the vector store and the model uses the new fact on the very next query — no retraining, and every claim carries a citation you can verify. Fine-tuning would bake yesterday's number into the weights and give you no source to check.

COMMON MISCONCEPTION

"Grounding makes hallucination impossible." It makes it *much rarer*, not impossible. The model can still misread a chunk or over-reach if retrieval returns the wrong passage — which is why the citation requirement matters: it lets you (or a checker) trace every claim back to its source and catch a bad retrieval.

IN ONE SENTENCE

"RAG scores your query by embedding it and ranking chunks by cosine similarity, then grounds the answer by pasting the top chunks into the model's prompt with a 'use only this + cite it' instruction — so the freshest facts reach the model through the context, never its frozen weights."

Amit Pandey · amit.s.pandey@gmail.com